University of Sheffield

School of Electrical and Electronic Engineering



From Behavioural Modelling to a Memristive Izhikevich-Inspired Neuron Circuit

Kaiyu Han

2026 May

Supervisor: Prof Merlyne M De Souza

A dissertation submitted in partial fulfilment of the requirements for the degree of Bachelors of Engineering (BEng) in Electrical and Electronic Engineering

Abstract

This project looks at the move from behavioural neuron modelling to a memristive Izhikevich-inspired circuit. In MATLAB, neuron models can produce clear spiking patterns by using ***equations***, ***reset rules*** and ***adaptive thresholds***. However, these operations are not as simple in a circuit, because they have to be produced through ***voltages, currents, components and feedback paths***. This gap was the main focus of the project.

The work started with MATLAB simulations of the ***Izhikevich, MAT and AMAT models***. These simulations were used to study typical firing behaviours such as tonic spiking, bursting and phasic responses, and to give a reference for the later circuit work. After this, a flux-controlled memristor subsystem was built and tested in ***Simulink/Simscape***. The memristor produced ***a pinched*** I–V ***hysteresis curve*** and showed clear set/reset behaviour, which confirmed that the memory-dependent part of the model was working at subsystem level.

The final stage was to connect the memristor into a ***MIZH-inspired neuron circuit***. Although the memristor branch affected the output response, a stable phasic-spiking waveform was not achieved. The output voltage rose to an unrealistic level, above $4000\ V$, which showed that the circuit was not correctly scaled. The result was closer to an RLC–$memristor$ transient than a true neuron spike. The main reasons were the missing ***reset/update path for the Izhikevich*** c ***and*** d ***parameters***, ***incomplete realisation of the nonlinear branch***, and ***input-scaling problems***. Overall, the project achieved a useful behavioural baseline and a verified memristor subsystem, but also showed that a full MIZH circuit needs further work on reset design, feedback and scaling.

Chapter 2 reviews the theoretical basis of neuron and memristive models. Chapter 3 presents the MATLAB behavioural simulations, the memristor subsystem verification, and the attempted MIZH circuit integration. Chapter 4 summarises the findings, limitations and future work, while Chapter 5 discusses the wider engineering context.

Individual Contribution

This dissertation is the result of my own work. I conducted the literature review, implemented the MATLAB simulations of the Izhikevich and AMAT neuron models, developed and verified the memristor subsystem, and explored the integration of this element into a memristive Izhikevich-inspired neuron circuit in ***Simulink/Simscape***. I was responsible for parameter selection, simulation testing, debugging, analysis of results, figure preparation, and the writing of the dissertation. Guidance from my supervisor and discussions with other researchers helped shape the direction of the project, but the modelling, simulation, interpretation, and presentation of the work were carried out by me unless otherwise stated. Any external sources, published models, or software tools used in this dissertation have been appropriately cited.

Acknowledgements

I would like to sincerely thank my supervisor, Professor Merlyne M. De Souza, for her guidance, encouragement, and thoughtful feedback throughout this project.

I am also grateful to Dr Song Xiaoyao for his helpful advice and discussions during the modelling and simulation stages of the project.

I would also like to thank Professor Mark Hopkinson, my second marker, for his helpful feedback and advice on the dissertation.

My thanks also go to the School of Electrical and Electronic Engineering at the University of Sheffield for providing the resources and environment needed for this work.

Finally, I would like to thank my family and friends for their patience, encouragement, and constant support throughout the project.

Contents

Abstract **1**
Individual Contribution 2
Acknowledgements 2
Contents **3**
List of Figures **5**
Chapter 1 **4**
1. Introduction 4
1.1 Background and Motivation 4
1.2 Problem Definition 4
1.3 Aims and Objectives 5
Chapter 2 **6**
2. Literature Review 6
2.1 Overview of Neuron Models 6
2.2 Traditional Neuron Models 6
2.2.1 Hodgkin-Huxley Model 6
2.2.2 Integrate-and-Fire Model 7
2.3 The Izhikevich Model 8
2.4 MAT and AMAT Models 11
2.5 Representative Spiking Behaviours 13
2.6 From Behavioural Models to Circuit Models 13
2.7 Memristor Theory and Behaviour 14
2.8 Memristive Neuron Circuits 15
2.9 Research Gap and Project Justification 18
Chapter 3 **18**
3. Methodology, Simulation Results and Discussion 18
3.1 Methodology Overview 18
3.2 Izhikevich Model Simulation and Results 19
3.3 MAT/AMAT Model Simulation and Results 22
3.4 Behavioural Reference Obtained from MATLAB Simulations 24
3.5 Memristor Subsystem Modelling and Verification 25
3.5.1 Why the Memristor Subsystem Matters 25
3.5.2 Selected Memristor Model and Governing Equations 25
3.5.3 Simulation Setup and Implementation 26
3.5.4 Verification of Memristor Behaviour 27
3.6 MIZH-Inspired Circuit Integration and Analysis 29
3.6.1 MIZH Concept and Intended Architecture 29
3.6.2 Mapping Neuron Dynamics into Circuit Elements 29

3.6.3 Integration of the Memristor into the Neuron Circuit 30
3.6.4 Numerical and Structural Challenges 31
Chapter 4 **33**
4. Conclusions 33
4.1 Summary of the Project 33
4.2 Main Findings 33
4.3 Limitations 34
4.4 Future Work 34
Chapter 5 **35**
Economic, Legal, Social, Ethical and Environmental Context 35
Appendix A **36**
A.1 Core MATLAB code used to generate the tonic spiking response of the Izhikevich neuron model 36
A.2 Core MATLAB Code for AMAT Spike Frequency Adaptation Simulation 36
A.3 Additional Izhikevich Model Simulation Results 38
A.4 Additional MAT/AMAT Model Simulation Results 40
A.5 MATLAB Function for Calculating Flux-Dependent Memristance 43
References **44**

List of Figures

Figure 1. Comparison of Different Models **[2].**

Figure 2. Parameter effects in the Izhikevich neuron model [1]

Figure 3. Izhikevich Model Parameter Sets for Different Neuron Types [1]

Figure 4. Twenty characteristic spiking patterns generated by the Izhikevich model [2]

Figure 5. I–V Hysteresis Characteristic of the Memristor Showing Set and Reset Transitions [6]

Figure 6. The neuron membrane circuits [6]

Figure 7. Tonic spiking behaviour of an Izhikevich neuron under step current input

Figure 8. Tonic bursting behaviour of an Izhikevich neuron under step current input

Figure 9. Class 2 Excitable Response of an Izhikevich Neuron under Ramp Current Input

Figure 10. Tonic bursting result in the MAT model using grouped spike-event detection

Figure 11. Initial spike-state visualisation of tonic bursting in the MAT model

Figure 12. Simulink/Simscape model used to test the flux-controlled memristor subsystem

Figure 13. Simulated pinched I–V hysteresis behaviour of the flux-controlled memristor

Figure 14. Simulink/Simscape implementation of the MIZH-inspired neuron circuit

Figure 15. Phasic spiking response of the MIZH model under step input [6]

Figure 16. Unscaled voltage response of the integrated MIZH-inspired circuit

Figure A.1 Phasic bursting behaviour of an Izhikevich neuron under step current input

Figure A.2 Mixed mode behaviour of an Izhikevich neuron under step current input

Figure A.3 Spike frequency adaptation behaviour of an Izhikevich neuron under step current input

Figure A.4 Class 2 behaviour of an Izhikevich neuron under ramp current input

Figure A.5 Phasic spiking behaviour of an Izhikevich neuron under step current input

Figure A.6 Mixed mode result using grouped spike-event detection

Figure A.7 Spike frequency adaptation result using grouped spike-event detection

Figure A.8 Class 1 result using grouped spike-event detection

Figure A.9 Class 2 result using grouped spike-event detection

Figure A.10 Phasic spiking result using grouped spike-event detection

Chapter 1

1. Introduction

1.1 Background and Motivation

Understanding how neurons generate different firing patterns has long been an important topic in computational neuroscience. Real neurons show rich and complex dynamics, but detailed biological models are often difficult to analyse and expensive to simulate**[1][2][5]**. For this reason, simplified neuron models are widely used to study the main features of spiking behaviour without the full complexity of biological detail**[1][2]**.

Among these simplified models, the Izhikevich model is especially useful because it can reproduce a wide range of firing patterns while remaining relatively simple**[1][2]**. This makes it a good starting point for studying neuronal dynamics at the modelling level. Adaptive-threshold models such as MAT and AMAT are also valuable, as they offer a different view of how spiking behaviour can be produced through ***threshold dynamics*** rather than only ***reset-based mechanisms***[3][4].

However, behaviour-level models do not directly show how these dynamics might be realised in circuit form. A model may produce the correct waveform in MATLAB, but this does not mean that it can be translated easily into an electrical system. This creates a gap between behavioural modelling and circuit implementation, which becomes more important when considering ***neuromorphic and low-power hardware***.

One possible route towards this transition is the use of ***memristive elements***. Because a memristor is nonlinear and has memory-dependent behaviour, it has attracted interest in neuron-inspired circuit modelling**[6]**. This motivates the present project, which began with behavioural simulations of neuron models and later moved towards the exploration of a memristive Izhikevich-inspired neuron circuit. The aim is not only to study firing behaviour, but also to examine how difficult it is to move ***from a mathematical description to a circuit-level realisation*.**

1.2 Problem Definition

Although neuron models can produce clear spiking patterns in MATLAB, this does not mean that the same behaviour can be easily built as a circuit. In code, a reset rule or a changing threshold can be written directly, but in a circuit these effects must come from real variables such as voltage, current, resistance and component connections.

This project therefore focuses on the gap between behavioural modelling and circuit-level simulation. The Izhikevich and AMAT models are first used to understand typical firing behaviours. After that, a memristor is tested separately to check whether it can show the expected memory effect and I–V hysteresis. Finally, the project explores how this memristor could be used in a MIZH-inspired circuit.

The main problem is not only whether a spike-like waveform can be produced, but whether the modelling process is reasonable and what difficulties appear during circuit implementation. These difficulties include ***parameter choice***, ***initial conditions*** and ***solver***

stability.

1.3 Aims and Objectives

The aim of this project is to study the transition from behavioural neuron modelling to memristive circuit-level simulation. The project first uses MATLAB simulations of the Izhikevich and AMAT models to understand ***basic spiking behaviour***, and then moves towards ***memristor modelling*** and the exploration of ***a MIZH-inspired circuit structure***.

The main objectives of this project are:

1. To review relevant neuron models, including the HH, Izhikevich, MAT and AMAT models, to understand the basic mechanisms behind neuronal firing**[1][2][3][4][5]**.

2. To simulate the Izhikevich and MAT/AMAT models in MATLAB and use them to build a behavioural reference for the later work (such as ***waveform similarity, burst count, and other relevant firing features***).

3. To develop and test a memristor subsystem in simulation, with focus on its memory-dependent behaviour and I–V characteristics**[6]**.

4. To explore how the memristor could be connected to a MIZH-inspired neuron circuit in ***Simulink/Simscape***.

5. To assess the feasibility of this approach and suggest what further work would be needed for a more complete MIZH circuit implementation.

Objective	Evidence / success criterion
Review neuron and memristive models	Explain why IZH, MAT/AMAT and memristor models are relevant to circuit-level neuron modelling
Simulate IZH and MAT/AMAT models	Reproduce representative firing behaviours using waveform shape, spike count or burst count
Verify memristor subsystem	Show pinched I–V hysteresis
Explore MIZH-inspired circuit integration	Compare circuit output with expected phasic-spiking reference
Assess feasibility	Identify scaling, reset/update, nonlinear branch and solver-related limitations

Table 1. Project Objectives and Success Criteria

This project is treated as a simulation-based feasibility study. The focus is not on producing a final hardware circuit, but on understanding the modelling steps, the successful parts of the simulation work, and the challenges involved in developing a memristive neuron circuit.

Chapter 2

2. Literature Review

This chapter reviews the main theories used in this project. It first introduces traditional neuron models, including the Hodgkin–Huxley model and the leaky integrate-and-fire model. It then discusses the Izhikevich model and the MAT/AMAT models, which were used in the first stage of the project for MATLAB-based behavioural simulations. The second part of the chapter introduces memristors and explains why they are relevant to circuit-level neuron modelling. The purpose of this review is not only to introduce each model separately, but also to explain how the project moved from behavioural neuron modelling towards circuit-level exploration.

2.1 Overview of Neuron Models

Neuron models are used to describe how neurons produce electrical spikes. Real biological neurons are complex, as their behaviour depends on ***ion channels***, ***membrane properties*** and ***time-dependent feedback processes*** **[1][2][5]**. A full biological description can be useful, but it is often too detailed for engineering simulation or large-scale modelling.

For this reason, different levels of neuron models have been developed. Some models try to stay close to the biological mechanisms, while others only aim to reproduce the main shape or timing of spikes **[2]**. This creates a trade-off between ***biological detail*** and ***simulation cost***. A detailed model may be more realistic, but it is usually harder to simulate. A simpler model is easier to use, but it may miss some important behaviours.

This trade-off is important for this project. The first stage of the work was based on MATLAB simulations of reduced neuron models, mainly the Izhikevich and MAT/AMAT models. They are still simple enough to implement, but they can show richer behaviours such as tonic spiking, bursting, adaptation and phasic firing etc. These behaviours then provided a reference before the project moved towards memristor modelling and the MIZH-inspired circuit.

2.2 Traditional Neuron Models

2.2.1 Hodgkin-Huxley Model

The Hodgkin-Huxley model is one of the most important models in computational neuroscience. It was developed from experimental work on the squid giant axon and describes the membrane voltage using sodium, potassium and leak currents **[5]**. These currents are controlled by voltage-dependent variables, which describe how ion channels open and close over time **[5]**.

The main strength of the Hodgkin-Huxley model is that it is closely linked to real biological mechanisms. It can describe the full action potential in much more detail than most reduced models **[1][2]**. Because of this, it is often treated as a reference point when discussing neuron models.

However, this level of detail also makes the model more difficult to use. ***It contains several coupled differential equations and many parameters.*** This means that it is not always suitable for ***large simulations*** or for ***quick engineering studies*** **[2][5]**. In this project, the Hodgkin-Huxley model was mainly used as background. It helped show why simpler models are useful, but it was not used as the main simulation model.

2.2.2 Integrate-and-Fire Model

The integrate-and-fire model is much simpler than the Hodgkin-Huxley model. It treats the neuron mainly as a ***leaking capacitor*** **[2]**. The membrane voltage rises or falls depending on the input current, and a spike is recorded when the voltage reaches a fixed threshold. After the spike, the voltage is reset **[2]**.

A common form of the model is:

$$\tau m \frac{dv}{dt} = -(v - v_{rest}) + RI \quad (1)$$

$$if\ V(t) \geq Vth, emit\ a\ spike \quad (2)$$

$$V \leftarrow Vreset \quad (3)$$

where v is the membrane voltage, τm is the membrane time constant, R is the input resistance, and I is the input current.

This model is easy to simulate and is useful when the exact spike shape is not important. However, it is limited because the threshold is fixed and the reset rule is very simple **(1)(2)(3)**. It cannot naturally reproduce many behaviours seen in real neurons, such as bursting, rebound spikes, adaptation or phasic firing **[2]**.

This limitation is one reason why more flexible reduced models are useful. The Izhikevich model sits between the detailed Hodgkin-Huxley model and the very simple integrate-and-fire model. It keeps the equations relatively simple, but can still reproduce a much wider range of firing patterns, as shown in ***Figure 1***.

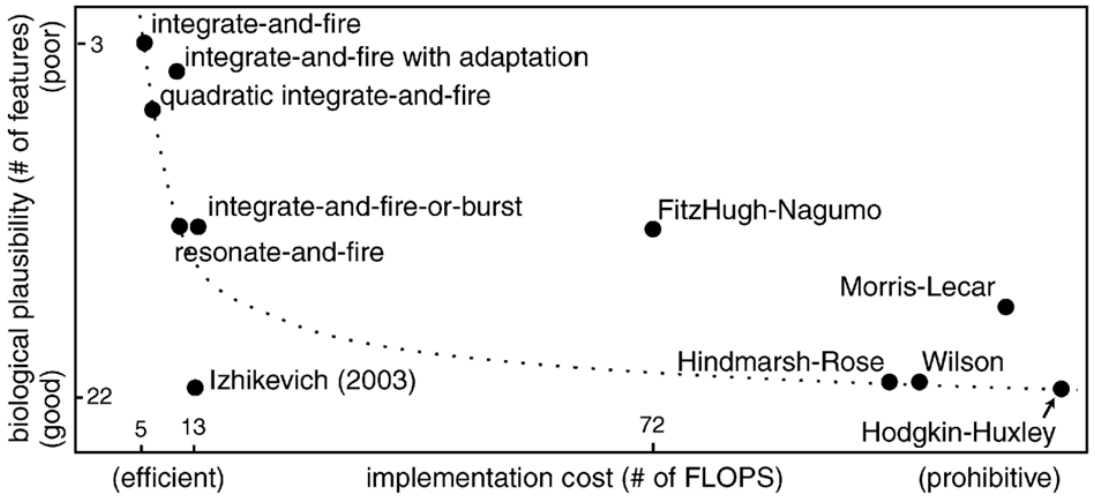


***Figure 1*. Comparison of Different Models [2].**

2.3 The Izhikevich Model

The Izhikevich model (2004) was introduced as a reduced model that can reproduce many firing patterns of cortical neurons without using a full conductance-based description **[1][2]**. Instead of modelling each ion channel separately, it uses two variables: the membrane potential v, and a slower recovery variable u.

The model is usually written as:

$$v' = 0.04v^2 + 5v + 140 - u + I \quad (4)$$

$$u' = a(bv - u) \quad (5)$$

When the membrane voltage reaches the spike cut-off value, normally 30 mV, the reset rule is applied:

$$v \geq 30 \Rightarrow (v \leftarrow c, u \leftarrow u + d) \quad (6)$$

In this model, v represents the ***membrane voltage***, while u acts as a ***slower feedback variable***. The variable u helps represent recovery effects, such as potassium activation and sodium inactivation, without modelling these biological processes in full detail **[1]**. The parameters a, b, c, and d control the speed of recovery, the coupling between voltage and recovery, and the reset behaviour after a spike **[1]**. For example, a sets the time scale of the recovery variable, b determines how strongly u responds to changes in voltage, and the pair (c, d) shapes the reset and the gradual build-up after each spike **[1]**, as shown in ***Figure 2*** and ***Table 2***.

Table 2. **Effect of Izhikevich model parameters on firing behaviour**

Parameter	**Meaning**	**Effect on firing**
a	recovery time scale	smaller a gives slower recovery
b	sensitivity of u to v	controls subthreshold coupling
c	reset voltage	determines post-spike voltage
d	recovery increment	controls post-spike adaptation

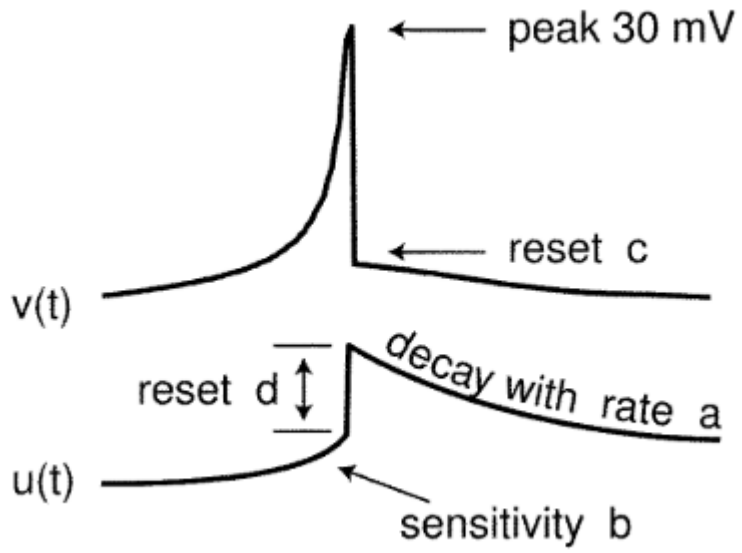


Figure 2**. **Parameter effects in the Izhikevich neuron model [1]

A major reason why this model is widely used is that small changes in these four parameters can generate many different firing patterns, as shown in ***Figure 3***. These include regular spiking, fast spiking, bursting, chattering, phasic firing and rebound responses **[1][2]**. This made it a useful starting point for the MATLAB part of this project.

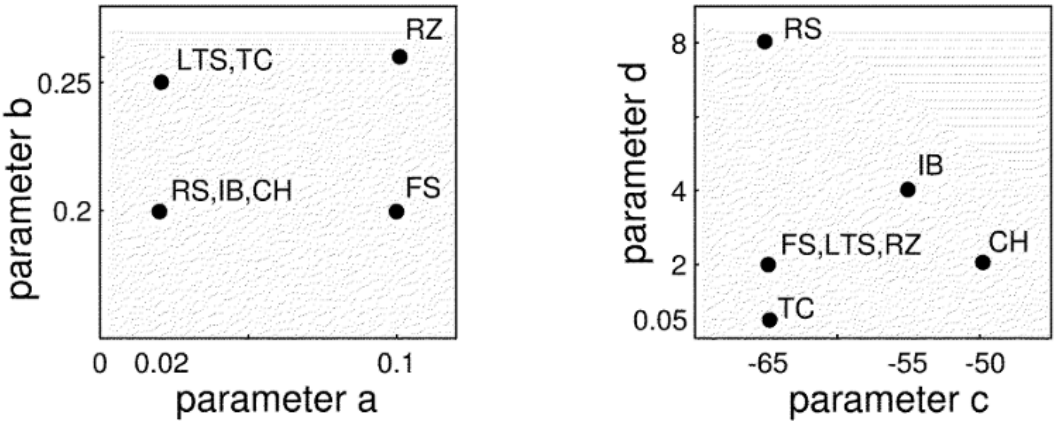


Figure 3**. **Izhikevich Model Parameter Sets for Different Neuron Types [1]

For the early stage of this work, the Izhikevich model provided a behavioural reference. It helped show how a relatively simple set of equations can generate complex firing patterns, as shown in ***Figure 4***. During implementation, the reset rule was also useful because it gave a clear way to detect and record spikes.

At the same time, the model has some limitations. The parameters do not have a direct ***one-to-one link*** with physical circuit components. Also, the reset rule is a mathematical operation rather than a natural electrical process. This became important later in the project, because producing a spike waveform in MATLAB is not the same as realising the same behaviour in a circuit. For this reason, the Izhikevich model was used as a starting point, rather than something that could be copied directly into circuit form.

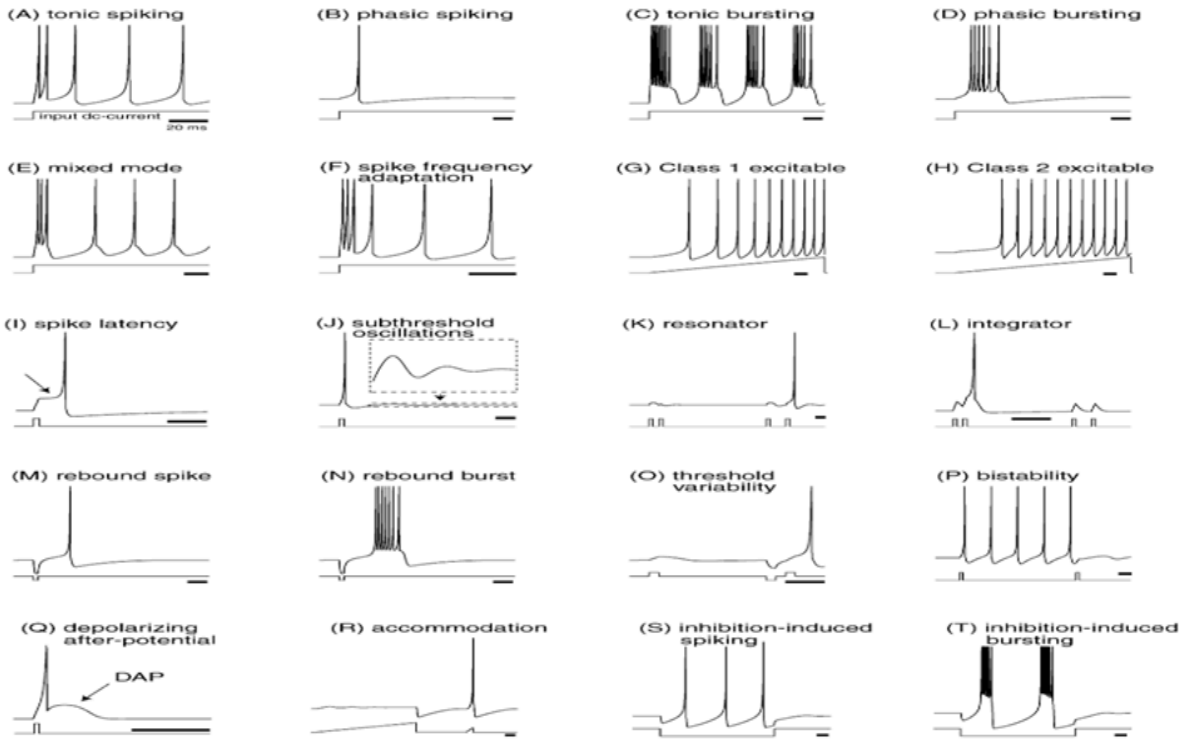


***Figure 4*. Twenty characteristic spiking patterns generated by the Izhikevich model [2]**

2.4 MAT and AMAT Models

The MAT model, or ***multi-timescale adaptive threshold model***, uses a different idea from the Izhikevich model. In the Izhikevich model, the spike behaviour mainly comes from the voltage equation, the recovery variable and the reset rule**[1][2]**. In MAT, the membrane voltage is simpler, ***but the threshold itself changes with previous spikes*** **[3]**.

The subthreshold voltage can be written in the form:

$$\tau_m \frac{dv}{dt} = -V + RI(t) \quad (7)$$

where τm is the membrane time constant, R is the input resistance, and $I(t)$ is the input current. A spike is recorded when the membrane voltage $V(t)$ reaches the dynamic threshold $\theta(t)$. Unlike the standard integrate-and-fire model, the voltage is not reset after a spike. Instead, ***the threshold is updated and then gradually returns towards its resting value*** **[3]**.

The adaptive threshold is written as:

$$\theta(t) = \sum_k H(t - t_k) + \omega \quad (8)$$

$$H(t) = \sum_{j=1}^{L} \alpha_j \, e^{\frac{-t}{\tau_j}} \quad (9)$$

Here, t_k represents previous spike times, ω is the resting threshold, and each pair α_j, τ_j (***threshold jump amplitude,decay time constant***) controls one exponential threshold component.The different time constants allow the threshold to change on more than one time scale. This is useful because a neuron's excitability does not only depend on the present input, but also on its recent firing history **[3].**

In the original MAT model, two threshold components are often enough to represent a fast and a slow part of the spike-triggered threshold. This allows MAT to reproduce behaviours such as tonic spiking, firing-rate adaptation, type-I and type-II responses, bistability and tonic bursting**[3][4]**. However, because the threshold depends mainly on previous spike times, the model is still limited for some transient behaviours.

Although MAT captures many behaviours, its threshold depends only on past spikes, so the model cannot generate the full variety of transient responses listed in Izhikevich's table of twenty firing patterns **[3]**. ***Yamauchi et al. (2011)*** therefore added a ***voltage-dependent term*** to the threshold, motivated by experiments showing that spike threshold correlates with the time derivative of the membrane potential **[3]**. In the augmented MAT (AMAT) model the threshold becomes:

$$\theta(t) = \sum_k H(t - t_k) + \beta \int_0^{\infty} K(s) \frac{dv}{dt}(t - s) ds + \omega \quad (10)$$

$$K(s) = s e^{-s/\tau_V} \quad (11)$$

In this expression, β controls how strongly the voltage-dependent term affects the threshold, and τ_V (a few milliseconds) sets the time scale of this effect. Here, $K(s)$ acts as an alpha-function kernel. It weights recent voltage changes and determines how this voltage-dependent effect is added to the adaptive threshold. The model still keeps the same leaky voltage equation and the same spike-triggered threshold term $H(t)$ but the extra voltage-dependent part makes the threshold respond to recent voltage changes as well **[3]**.

This small addition makes AMAT more flexible than MAT. It can reproduce behaviours such as phasic spiking, phasic bursting, rebound responses, latency and threshold variability**[3][4]**. For the early MATLAB stage of this project, AMAT was useful because it showed that complex firing patterns can be produced through adaptive threshold dynamics, rather than only through the reset mechanism used in the Izhikevich model.

MAT mainly represents spike-history-dependent adaptation, whereas AMAT adds a voltage-history-dependent threshold term. Therefore, AMAT is better suited to transient behaviours, but it also introduces a threshold mechanism that is harder to translate into a ***simple physical circuit***.

In the final direction of this dissertation, MAT and AMAT are not treated as the main circuit models. Their role is mainly to support the first stage of the work, where behavioural neuron models were studied in MATLAB. They helped build an understanding of threshold adaptation and firing behaviour before the project moved towards memristor modelling and the MIZH-inspired circuit exploration.

2.5 Representative Spiking Behaviours

The early MATLAB work used several representative spiking behaviours as reference cases. These behaviours are mainly named according to the voltage response produced by the neuron, as shown in ***Figure 4***. For example, ***tonic spiking*** means that the neuron continues to fire while the input is applied, while ***phasic spiking*** means that it fires mainly at the start of the input. ***Bursting*** refers to groups of closely spaced spikes, so ***tonic bursting*** describes repeated bursts during a sustained input, and ***phasic bursting*** describes a burst at the beginning of stimulation**[1][2].**

Other behaviours describe more specific features of the response. ***Spike-frequency adaptation*** means that the firing rate gradually slows down during a sustained input. ***Rebound spiking*** or ***rebound bursting*** occurs after an inhibitory input is removed. **Resonator** behaviour means that the neuron responds more strongly to inputs with a suitable timing or frequency, while ***threshold variability*** means that the same input may or may not cause a spike depending on the previous state of the neuron **[1][2]**.

In this project, these behaviours were used as a behavioural reference rather than as the final aim of the dissertation. They helped test whether the MATLAB models could reproduce more than simple repeated spikes, and also provided a basic idea of what neuron-like behaviour should look like before moving towards memristor modelling and MIZH-inspired circuit exploration. ***In the later simulations, these behaviours were assessed using waveform shape, spike count, burst count, and whether the response was tonic or phasic.***

2.6 From Behavioural Models to Circuit Models

The early models discussed above are mainly behavioural models. They are written as equations and can be simulated directly in MATLAB. This makes it possible to ***tune parameters***, ***apply input currents*** and observe ***the resulting spike patterns***.

However, a behavioural model does not automatically become a circuit model. In MATLAB, a reset can be applied instantly by writing a condition in code. A threshold can also be

defined directly as a variable. In a physical or circuit-level model, these effects need to come from voltages, currents, components and connections. This makes the problem much harder.

For example, the Izhikevich model uses a quadratic voltage term and a hard reset rule **[1][2]**. These are simple to write in code, but they are not easy to match directly to standard circuit components. Similarly, AMAT uses a dynamic threshold that depends on spike history and voltage change. This is useful for modelling, but implementing the same idea in circuit form would need extra memory or feedback mechanisms.

This is why the later part of the project moved towards memristive modelling. A memristor has memory-dependent behaviour, so it may offer a possible way to represent some of the history-dependent effects seen in neuron models **[3][4][6]**. The aim was not to build a complete biological neuron circuit, but to explore whether a memristive element could be used as part of an Izhikevich-inspired circuit structure.

Behavioural model operation	**Easy in MATLAB because…**	**Circuit-level requirement**
Voltage reset $v \leftarrow c$	direct ***if-statement***	switch, comparator or feedback path
Recovery update $u \leftarrow u + d$	direct variable update	storage element
Dynamic threshold $\theta(t)$	stored as a variable	memory path
Voltage-history term	numerical convolution/integration	integrator or memristive state
Memristance $M(\phi)$	equation-based update	physically compatible variable resistance

Table 3. Behavioural Model Operations and Circuit-Level Requirements

2.7 Memristor Theory and Behaviour

A memristor is a circuit element whose resistance depends on its previous electrical history. Unlike a normal resistor, its resistance is not fixed. ***It changes according to the past voltage applied to it*** **[6]**. This gives the memristor a memory effect.

A simple way to describe a memristor is to treat it as ***a variable resistor with an internal state*** **[6]**. The internal state changes over time, and the resistance depends on that state. Because of this, the same applied voltage may lead to different currents depending on the previous condition of the device **[6]**.

A common sign of memristive behaviour is ***a pinched hysteresis loop*** in the current-voltage curve. If a periodic input voltage is applied, the I-V curve does not look like the straight line of a normal resistor **[6]**. Instead, it forms a loop, as shown in ***Figure 5***. This shows that the current depends not only on the present voltage, but also on the previous state of the device.

This behaviour is one reason why memristors are interesting for neuron-inspired circuits **[6]**. Neurons also have history-dependent behaviour **[3][4]**. For example, their response to an

input can depend on previous firing, recovery, adaptation and threshold changes. A memristor cannot represent all of this by itself, but its memory effect makes it a useful element to investigate.

In this project, the pinched *I–V* loop was used as the main check of the memristor subsystem. If the simulated device behaves like a normal resistor, the *I–V* curve should be almost a straight line. If it behaves as a memristive element, the current should depend on the past voltage history and the *I–V* curve should form a loop, as shown in ***Figure 5***. This does not prove that the full neuron circuit will work, but it is a necessary first step before using the memristor inside the MIZH-inspired circuit.

A pinched I–V loop verifies that the memristor subsystem is memory-dependent, but it does not by itself guarantee that the full MIZH circuit will generate correct neuron-like spiking.

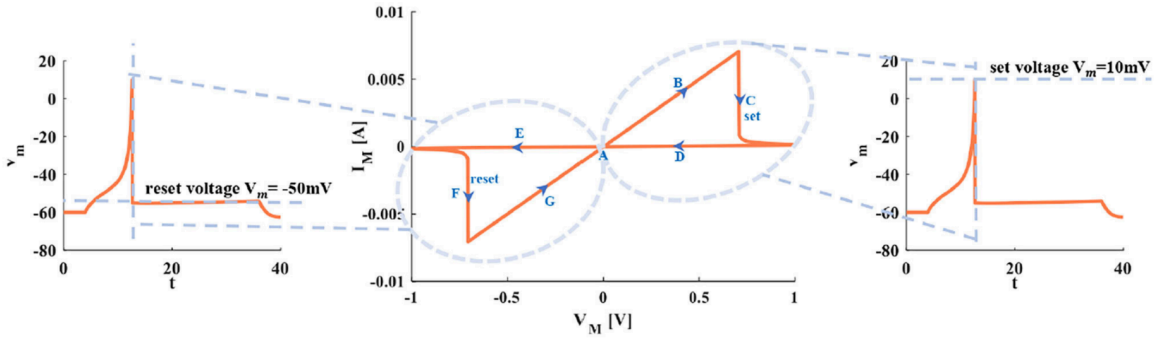


Figure 5. I–V Hysteresis Characteristic of the Memristor Showing Set and Reset Transitions[6]

2.8 Memristive Neuron Circuits

Memristive neuron circuits try to use the memory effect of a memristor to reproduce some neuron-like electrical behaviour. This is different from a purely mathematical neuron model, because the firing behaviour is now linked to circuit variables such as voltage, current and resistance. Fang et al. proposed a memristive Izhikevich model by introducing a ***flux-controlled memristor*** into the IZH framework **[6]**.

In their model, the memristor is treated as a variable resistance related to the ionic channel **[6]**. Its resistance depends on the flux $\phi(t)$, and is written as a piecewise function:

$$M(\varphi(t)) = \begin{cases} 20000 & \varphi(t) < -0.75 \\ \sqrt{-3.98 \times 10^8 \varphi(t) + 10^8} & \varphi(t) \geq -0.75 \text{ and } \varphi(t) < 0.25 \\ 100 & \varphi(t)) \geq 0.25 \end{cases} \quad (12)$$

$$\phi(t) = \int_{t_0}^{t} v(t)\, dt \quad (13)$$

In this equation (13), $\phi(t)$ is the time integral of the voltage across the memristor between t_0 and t, so it reflects the voltage history of the device.

This means the memristor can switch between ***a high-resistance state*** and ***a low-resistance state***. When a ***sinusoidal voltage*** is applied, the memristor produces a pinched *I–V* hysteresis loop. This loop is important because it shows that the current is not only determined by the present voltage, but also by the previous state of the device. In the paper, the authors (***Figure 3***) relate this set/reset process to the rise and fall of a neuronal action potential **[6]**.

The circuit interpretation is then built from the original Izhikevich model. ***The membrane potential*** v ***is treated as the capacitive voltage*** u_c***, and the recovery variable*** u ***is treated as the inductive current*** i_L ***as shown in Figure 6.*** The nonlinear term in the voltage equation

$$f(v) = a_1 v^2 + a_2 v + a_3 \quad (14)$$

is interpreted as a current-voltage relationship. For the IZH circuit, this is connected with a nonlinear resistor **[6]**. For the MIZH circuit, the fixed resistance is replaced by the memristance M **[6]**. The MIZH model is written as:

$$u_c' = a_1 u_c^2 + a_2 u_c + a_3 - a_4 i_L + a_5 I_{ext} \quad (15)$$

$$i_L' = R\left(\frac{u_c}{M} - i_L\right) \quad (16)$$

$$\phi' = u_c = -M i_M = -M(a_1 v^2 + a_2 v + a_3) \quad (17)$$

These equations show why the model is more difficult than the original IZH model. The voltage u_c, current i_L, flux ϕ ,and memristance M are all linked together. Therefore, a small change in one part of the circuit can affect the whole response.

Here, $a1$, $a2$, and $a3$ describe the quadratic, linear and constant voltage terms inherited from the original Izhikevich voltage equation, while $a4$ controls the effect of the recovery current

$iL$and $a5$ scales the external input current $Iext$. ***These coefficients are not the same as the Izhikevich parameter*** a . In the original Izhikevich model, a mainly controls the time scale of the recovery variable, whereas in this circuit form the recovery dynamics are represented through the $Rs - L$ branch. Therefore, $a1 - a5$ describe how the capacitor voltage is driven by nonlinear voltage dynamics, recovery feedback and external stimulation.

Fang et al. tested the IZH circuit and the MIZH circuit using a square-wave current input of 20 nA. In the time range from 100 ms to 300 ms, the IZH circuit produced 19 spikes, while the MIZH circuit produced 38 spikes. This suggests that the memristive version can produce a higher firing frequency under the same input conditions **[6]**. The paper also reports that the MIZH model can reproduce 23 spiking patterns and can show stronger spike-frequency adaptation in some cases compared with the original IZH model **[6]**.

Although Fang et al. demonstrate richer firing behaviour in the MIZH model, reproducing this result in ***Simscape*** is not automatic because the published equations assume an idealised memristive relationship, while Simscape requires ***explicit electrical compatibility***, ***initialization***, and ***solver-stable component connections***.

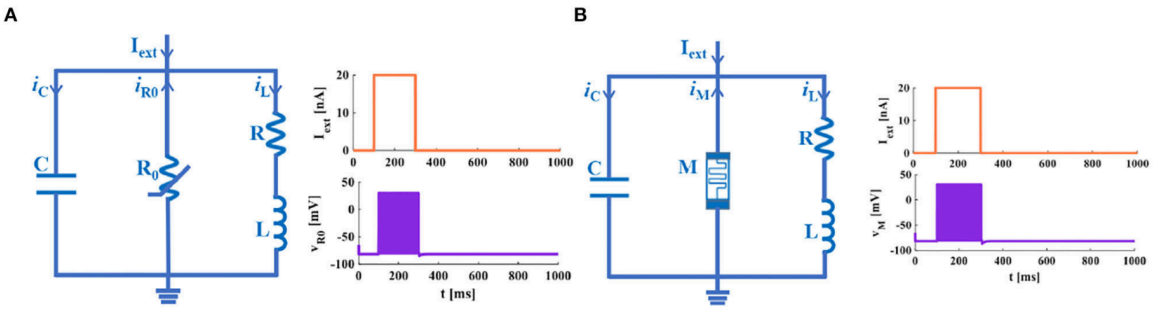


Figure 6. The neuron membrane circuits. (A) The IZH circuit model and its response to a single pulse current. (B) The MIZH circuit model and its response to a single pulse current [6]

The work by Fang et al. is important because it suggests a possible route from the Izhikevich equations to a memristive neuron circuit. In this approach, the membrane voltage is represented by a capacitive voltage, the recovery variable is linked to an inductive branch, and the fixed resistance is replaced by a memory-dependent memristive element. This makes the model relevant to neuromorphic circuit design, since it attempts to generate neuron-like firing behaviour through circuit variables rather than only numerical equations.

For this project, this paper is important because it gives a clear route from the Izhikevich equations to a memristive circuit structure. It also shows why the memristor was first tested as a separate subsystem. Before building a full MIZH circuit, the memristor needs to show the expected I–V hysteresis and switching behaviour. At the same time, the equations also

show why full circuit integration is challenging. The MIZH model is not just the original IZH model with one component added; the memristor introduces an extra dynamic state, which makes the simulation more sensitive to parameters, initial conditions and solver settings.

2.9 Research Gap and Project Justification

From the papers reviewed above, it is clear that models such as the Izhikevich model and AMAT are useful for MATLAB simulation. They can produce different firing behaviours, without using a very detailed biological model. In MATLAB, reset rules, threshold changes and state variables can be written directly in code. In a circuit, these effects have to come from voltages, currents, resistors, capacitors and other connected parts.

The MIZH model is useful for this project because it tries to add a memristor into an Izhikevich-type circuit. The memristor is interesting because its resistance depends on its past voltage, so it may help to represent some memory-like behaviour. However, rebuilding this model in ***Simulink/Simscape*** is not just copying the equations from the paper. The units, initial values, solver settings and circuit connections all affect the result.

For this reason, the main gap in this project is the step between the published MIZH equations and a working Simscape circuit. The project first uses Izhikevich and MAT/AMAT simulations as a behavioural reference, then tests the memristor by itself, and finally tries to connect it into a MIZH-inspired circuit. ***The aim is not to claim that a complete hardware neuron has been built, but to show what works, what does not work, and why this step from equations to circuit simulation is difficult.***

Chapter 3

3. Methodology, Simulation Results and Discussion

3.1 Methodology Overview

Before presenting the detailed results, the overall simulation route is summarised in ***Table 4***. The project was carried out in stages, starting from MATLAB behavioural neuron models, then moving to memristor verification, and finally to the attempted MIZH-inspired circuit integration. This was done so that the circuit result could be judged against a clear behavioural reference, rather than only by whether a spike-like waveform appeared.

Stage	Work carried out	Main result checked	Purpose
1	Izhikevich model simulated in MATLAB	$v(t)$, spike count and waveform shape	To build a behavioural reference for typical firing patterns

2	MAT and AMAT models simulated in MATLAB	Dynamic threshold, spike events and burst events	To study firing behaviour produced by adaptive threshold dynamics
3	Flux-controlled memristor tested in Simulink/Simscape	$(I)-(V)$ hysteresis, flux and memristance change	To verify that the memristor subsystem shows memory-dependent behaviour
4	Memristor connected into a MIZH-inspired circuit	$(Vout(t))$, voltage scale and similarity to phasic spiking	To test whether the memristor could support circuit-level neuron behaviour
5	Circuit output compared with behavioural reference	Successful parts, failed parts and modelling limitations	To assess the feasibility of the MIZH-inspired approach

Table 4**. **Overview of the simulation methodology used in this project

3.2 Izhikevich Model Simulation and Results

The Izhikevich model was implemented first, as it gave a clear and simple starting point for the simulations. The model uses two state variables: the membrane voltage v and the recovery variable u. The equations were solved using an **explicit Euler method** ($T = 1000ms \; and \; dt = 0.1ms$) . A complete version of the code is provided in the **Appendix A.1 and A.2**.

At each time step, v and u were updated from the model equations. When the membrane voltage reached the spike cut-off value of 30 mV, a spike was recorded and the reset rule was applied:

$$v \geq 30 \Rightarrow (v \leftarrow c, u \leftarrow u + d)$$

This reset rule was important because it allowed the model to generate repeated spikes without needing to simulate the full shape of each action potential. The parameter sets used in these simulations were taken from Izhikevich's original paper, as shown in ***Figure 3***.

Step currents (input from $200ms$ to $800ms$) were mainly used as input signals, since they provide a simple way to test whether the model can produce sustained firing. ***Different parameter sets were then used to reproduce several typical firing patterns.*** Due to space

limitations, only selected representative Izhikevich simulation results are presented in this section, while additional representative figures are provided in the **Appendix A.3**.

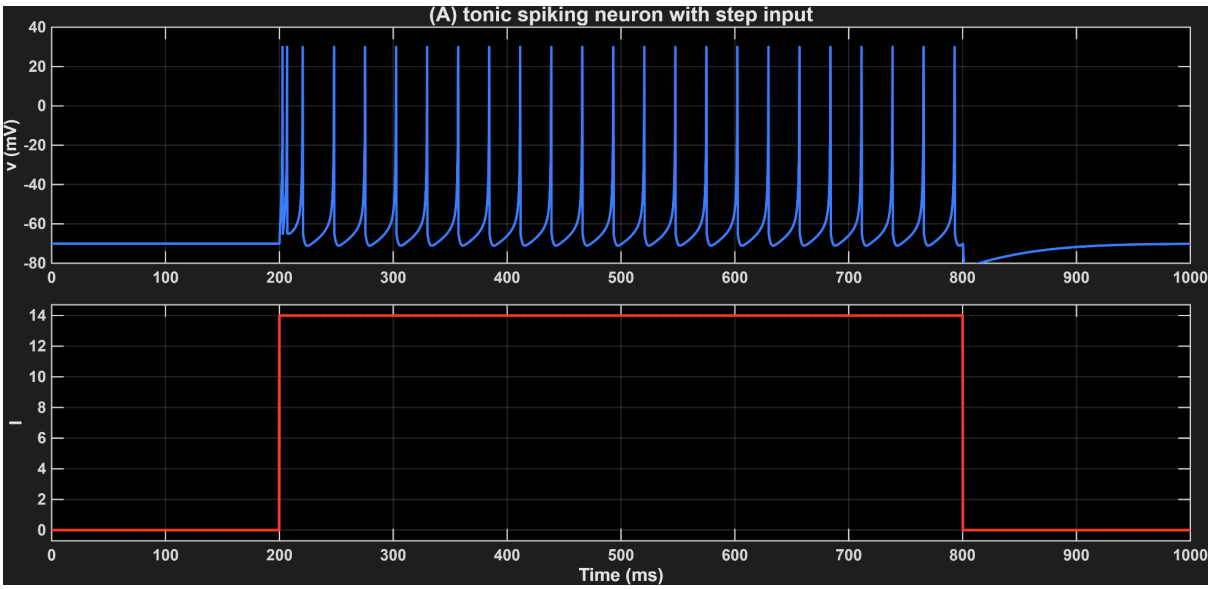


Figure 7. **Tonic spiking behaviour of an Izhikevich neuron under step current input**

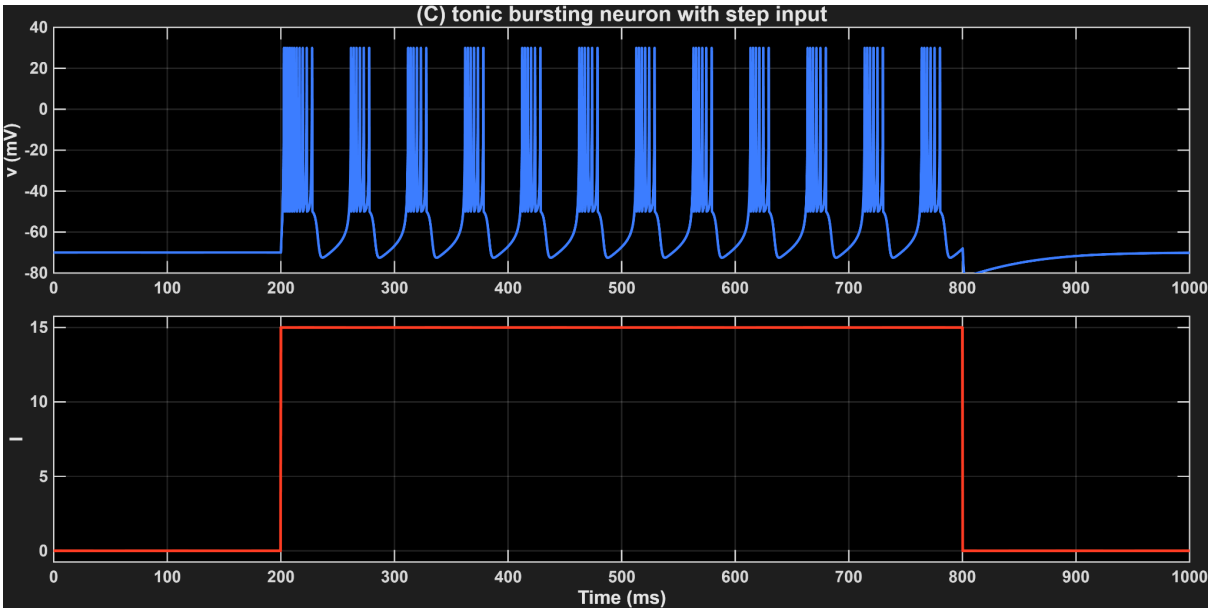


Figure 8. Tonic bursting behaviour of an Izhikevich neuron under step current input

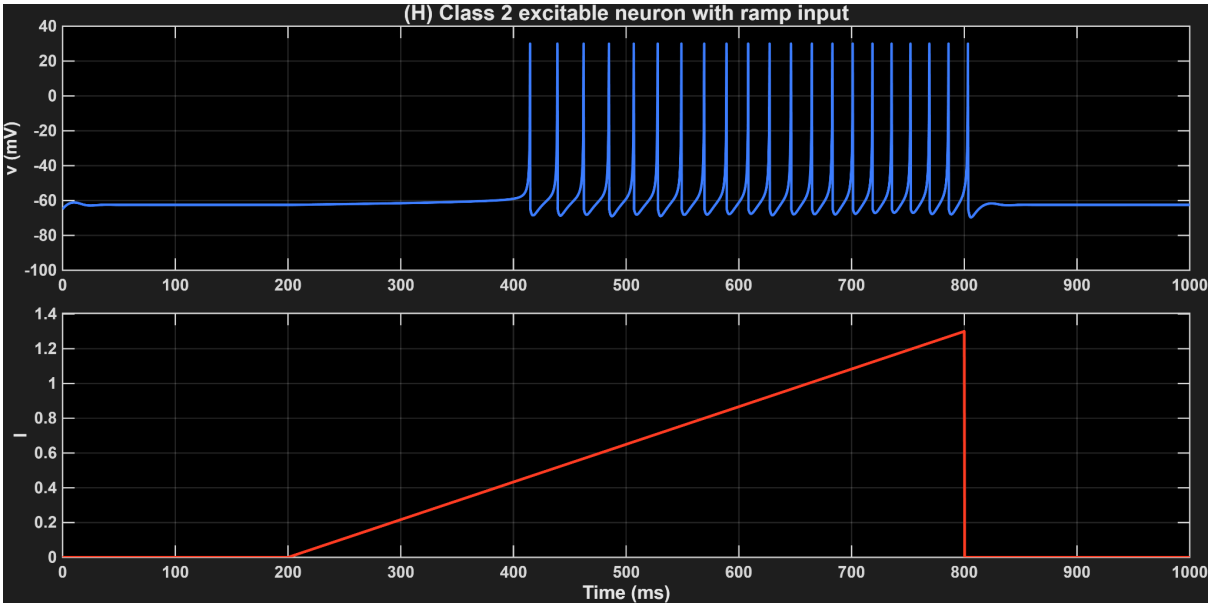


***Figure 9**. Class 2 Excitable Response of an Izhikevich Neuron under Ramp Current Input*

This implementation gave the project a stable behavioural baseline. It also helped show how strongly the firing pattern depends on the parameters a, b, c, and d . In particular, the reset values c and d had a clear effect on whether the output behaved more like regular spiking, bursting or chattering.

One clear point from these simulations is that the reset part of the model changes the firing pattern a lot. The voltage is first allowed to reach 30 mV, and then it is reset by c, while the recovery variable u is increased by d. When c is lower, the voltage is pushed further away from the next spike, so the neuron needs more time to fire again. This is why the regular spiking case shows a slower firing rate after the first few spikes. For bursting cases, c is usually set closer to the spiking region, so the next spike can happen more quickly. Several spikes can then appear close together before u becomes large enough to slow the response down. And also the parameter a controls the time scale of the recovery variable u . A smaller a means that u changes more slowly, so the neuron keeps some memory of its previous activity for a longer time. The parameter b controls how strongly u responds to the membrane voltage v before a spike occurs. ***This indicated that the different waveforms were not just produced by changing numbers, but by the way the reset voltage and recovery variable work together after each spike.***

The results were not judged only by whether spikes appeared. They were compared using several simple features, including spike count, overall similarity of the voltage waveform.This made the MATLAB results more useful as a behavioural reference for the later circuit work.

Case	**Input type**	**Parameter set**	Main result	Figure
Tonic spiking	step current	a, b, c, d = 0.02 0.20 -65 6	repeated regular spikes	***Fig. 7***
Tonic bursting	step current	a, b, c, d = 0.02 0.20 -50 2	repeated burst	***Fig. 8***

			groups	
Class 2 excitability	ramp current	a, b, c, d = 0.20 0.26 -65 0	sudden onset firing at threshold	***Fig. 9***

Table 5. Izhikevich Simulation Cases Used as Behavioural References

3.3 MAT/AMAT Model Simulation and Results

After the Izhikevich model had been tested, the MAT and AMAT models were implemented. These models use a different structure. The membrane voltage follows a leaky integration equation, but the firing threshold is not fixed. ***Instead, the threshold changes with previous spikes and then relaxes over time.***

In the MAT model, the adaptive threshold is built from spike-triggered components with different time constants. This allows the model to include both short-term and longer-term threshold changes. A spike is recorded when the membrane voltage crosses the dynamic threshold.

The AMAT model extends MAT by adding a voltage-dependent term to the threshold. This means the threshold is affected not only by spike history, but also by recent changes in membrane voltage. This made the AMAT model more flexible, especially for behaviours such as phasic firing, rebound responses and bursting.

For the MAT and AMAT simulations, a time step of $dt = 0.05ms$ was used, and each case was simulated for $400\,ms$. The membrane time constant was set to $\tau m = 10\,ms$, with $R = 50Mohms$. The threshold had two adaptive parts: a fast part with $\tau 1 = 10\,ms$ and a slow part with $\tau 2 = 200\,ms$. This allowed the threshold to change quickly after a spike, while still keeping some slower effect from earlier spikes. In the AMAT model, $\tau V = 5\,ms$ was used for the voltage-dependent term. The values of α1, α2 and β were then changed for different firing behaviours.

During the AMAT and MAT simulations, a small numerical issue was found in ***the bursting cases***. If every threshold crossing was treated as a separate event, the measured number of bursts could become sensitive to the time step. With a smaller time step, the simulation gives more sampled points around the threshold, so one physical burst may appear as several separate crossings in the recorded data. In that case, the burst count would partly reflect the **numerical resolution** rather than **the actual firing behaviour**.

To reduce this problem, the burst detection was based on the timing of spike events instead of counting every threshold crossing directly. After the spike times were detected, neighbouring spikes were grouped together when their **inter-spike interval** was shorter than a selected limit (For example $6ms$). **A group was only counted as a burst if it contained more than one spike.** In this way, several closely spaced spikes were treated as one burst event, while isolated spikes were kept as single-spike responses. The effect of this method can be seen by comparing ***Figures 10*** and ***11***, where the grouped approach avoids splitting one burst into several separate numerical events.

This gave a more stable comparison between tonic bursting, phasic bursting and mixed-mode firing, and reduced the effect of numerical resolution on the AMAT results.

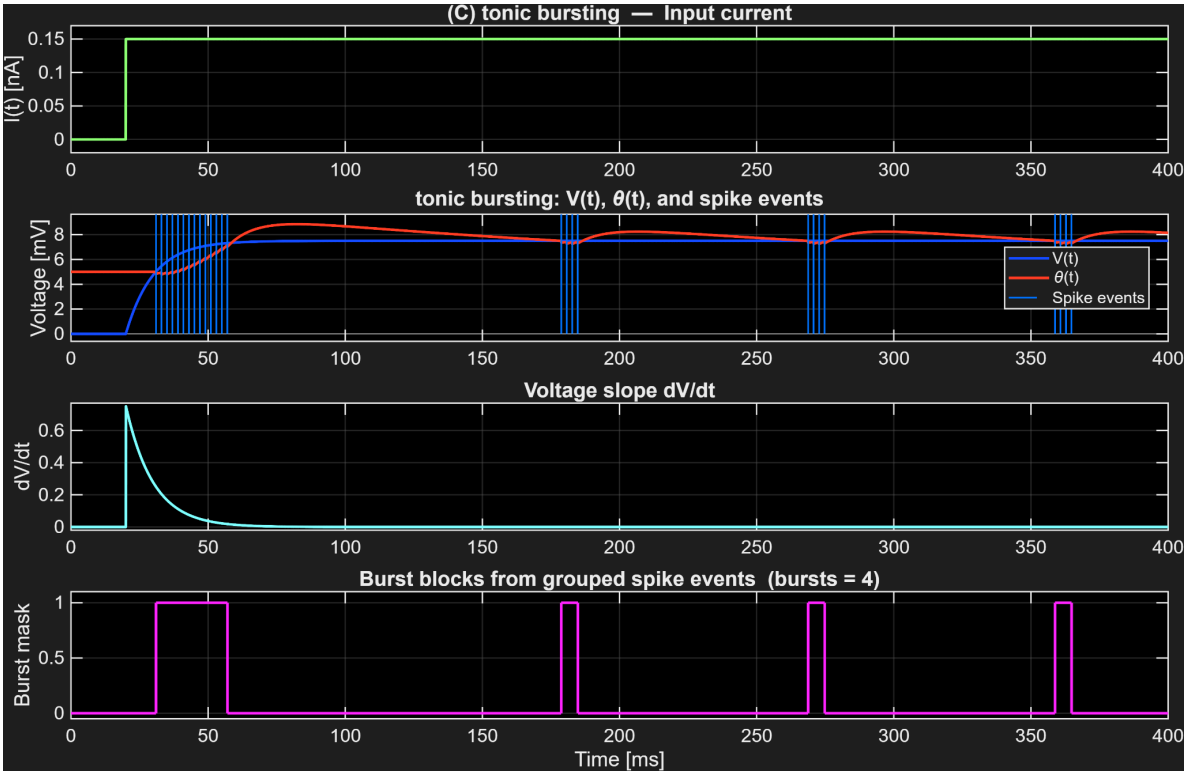


Figure 10. Tonic bursting result in the MAT model using grouped spike-event detection

Item	Value / method
Simulation time	400 ms
Time step	0.05 ms
Membrane time constant	10 ms
Input resistance	50 $M\Omega$
Fast threshold time constant	10 ms
Slow threshold time constant	200 ms
Voltage-history time constant	5 ms
Burst grouping criterion	neighbouring spikes grouped if ISI < 6 ms

Table 6. MAT/AMAT Simulation Settings and Detection Method

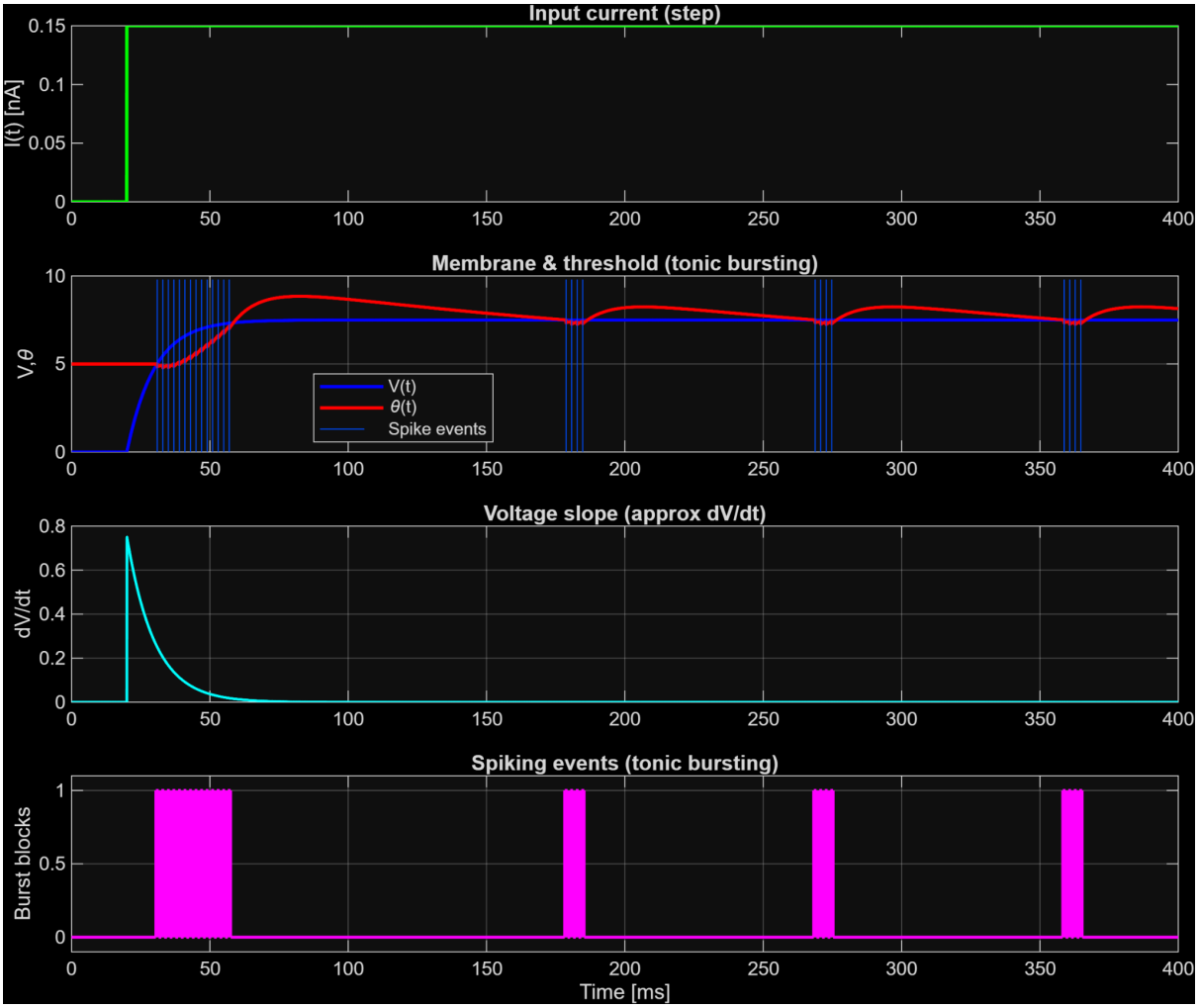


Figure 11. Initial spike-state visualisation of tonic bursting in the MAT model

Figure 11 shows the raw spike-state visualisation, while Figure 10 shows the grouped burst-counting result. The grouped method is therefore used in later comparisons because it is less sensitive to the ***sampling resolution***.

3.4 Behavioural Reference Obtained from MATLAB Simulations

The MATLAB work was useful because it showed that the firing pattern is not only decided by the input current. The internal variables of the model also have a strong effect on the final waveform. In the Izhikevich model, the recovery variable and the reset rule control how quickly the neuron can fire again after a spike. This explains why small changes in the parameter set can change the result from regular spiking to bursting or adaptation.

The MAT and AMAT results showed the same idea from another direction. In these models, the voltage is not reset after each spike. Instead, the threshold moves after previous spikes and then slowly returns. This made it clearer that spiking behaviour can also be controlled by threshold history, not only by the membrane voltage itself. For the AMAT model, the voltage-dependent threshold term added another effect, because the threshold could also respond to recent changes in the voltage.

The comparison between the models also helped to define what should be checked in the later circuit work. A useful neuron-like response should not be judged only by whether a spike appears. The spike count, overall waveform similarity all need to be considered. ***This is important because two plots may look similar at first, but they may come from different internal mechanisms.***

One main lesson from this stage is that MATLAB makes some operations easy. A reset rule, a moving threshold, or a spike-triggered update can be written ***directly*** in code. In a circuit model, these same effects must come from ***voltages, currents, components and feedback paths.*** This is why the later MIZH-inspired circuit is harder than the behavioural simulations. The memristor was therefore tested as a possible way to add memory-dependent behaviour, ***but the MATLAB results also show that a memristor alone does not guarantee that the full neuron response will be reproduced.***

Overall, this stage gave a behavioural reference for the rest of the project. It showed what kinds of firing behaviour should be expected, and it also showed why moving from equations to circuit simulation is not straightforward.

3.5 Memristor Subsystem Modelling and Verification

3.5.1 Why the Memristor Subsystem Matters

In the MIZH-inspired circuit, the memristor is expected to provide the memory-dependent part of the model, rather than acting as a normal fixed resistor. For this reason, it had to be tested on its own before being connected into the full neuron circuit. ***If the memristor cannot show the expected change in resistance and I–V hysteresis, then the later circuit result would not be reliable.*** Testing the subsystem first also makes the work easier to check, because problems with units, initial values or solver settings can be found before the whole circuit becomes too complicated. Therefore, verifying the memristor subsystem is an important step before discussing the full MIZH simulation.

3.5.2 Selected Memristor Model and Governing Equations

The memristor model used in this project is based on the flux-controlled memristor used in the MIZH paper. It was chosen because it links directly to the MIZH circuit idea, where the memristor is not just a normal resistor, but a resistance that changes with its past voltage. This makes it suitable for testing memory-like behaviour before it is connected into the full neuron circuit.

In this model, the main state variable is the flux $\phi(t)$. It is defined as the time integral of the voltage across the memristor, as described by ***Equation (13)***. This means that the memristor does not only depend on the present voltage. It also depends on what voltage has been applied before. The memristance $M(\phi)$ then changes according to this flux value.

The current-voltage relationship can be written as:

$$i_M(t) = \frac{v_M(t)}{M(\phi)} \tag{18}$$

So, when $M(\phi)$ changes, the same applied voltage can give a different current. This is the main reason why the $I-V$ curve can form a hysteresis loop instead of a straight resistor line.

In the selected model, the memristance has a **high-resistance region**, a **low-resistance region**, and a **transition region** between them as shown in ***Figure 5***. The key parameters are the two limiting resistance values and the flux boundary values, which decide when the device moves between these states.

3.5.3 Simulation Setup and Implementation

The memristor subsystem was built in MATLAB/Simulink/Simscape. A sinusoidal voltage signal was used as the input excitation, ***with an amplitude of*** $1V$ ***and a frequency of*** $1\ Hz$. This input was chosen because it is very close to the setting used by Fang et al. **[6]**, where a sinusoidal voltage is applied to check the $I-V$ hysteresis behaviour of the memristor. In the Simscape model, the input signal drives the electrical part of the circuit through a controlled voltage source. ***A current sensor was used to measure the current through the memristor, and a voltage sensor was used to measure the voltage across it.*** A ***Transport Delay*** block with a delay value of 1 was also added in the feedback path to avoid ***an algebraic loop*** between the voltage measurement, flux calculation, memristance update and variable resistor.

The measured voltage was integrated to get the flux $\phi(t)$. This value was then sent to the memristance block to calculate $M(\phi)$, which controlled the variable resistor in ***Simscape***. In this way, the resistance changed with the past voltage instead of staying fixed. The voltage and current were exported to the MATLAB workspace for plotting the $I-V$ curve.

The solver was set to **ode15s**. This was chosen because the circuit contains a state-dependent resistance and an integration path, which can make the model **numerically sensitive**. Compared with a simpler non-stiff solver, **ode15s** gave a more stable result for this Simscape circuit. This made the simulation more **stable** before the memristor was used in the full MIZH circuit.

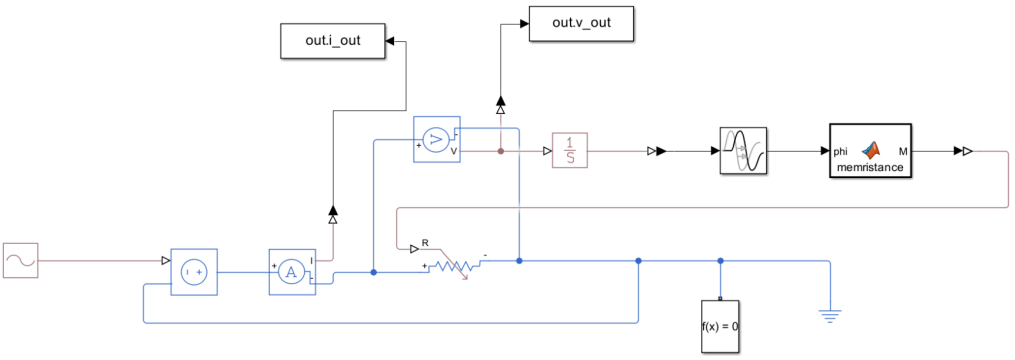


Figure 12. Simulink/Simscape model used to test the flux-controlled memristor subsystem

3.5.4 Verification of Memristor Behaviour

The memristor subsystem was mainly checked through its I–V curve. In Fang et al.'s paper, the expected behaviour of the flux-controlled memristor is a pinched ***hysteresis loop*** passing through the origin, with clear set and reset parts during one voltage cycle. This was used as the reference for this test.

As shown in ***Figure 13***, the simulated result gives a similar overall shape. The curve does not behave like a fixed resistor, because the current does not follow the same path when the voltage increases and decreases. Instead, the loop closes near the origin and shows two switching regions on the positive and negative sides. This means that the current is affected by the previous voltage history, which is the main feature expected from a memristor.

There is still a ***small sharp peak*** near the positive switching point. This is most likely caused by the piecewise memristance model, as described by ***Equation (12)*** . When the flux crosses the boundary value, the resistance changes very quickly, so the current also jumps for a short time. Therefore, this peak is treated as a switching transient, not as a separate steady behaviour. A similar sharp transition can also be seen in the set/reset regions of Fang et al. 's result, although the exact shape is not identical due to differences in ***numerical implementation***, ***solver behaviour*** in this project.

Overall, the subsystem gives the main behaviour needed for this project. It shows a pinched I–V loop, clear set/reset switching, and a response that depends on the internal state rather than only the present voltage. This does not prove that the full MIZH circuit will work, but it gives enough confidence to use the memristor block in the next stage of the circuit exploration.

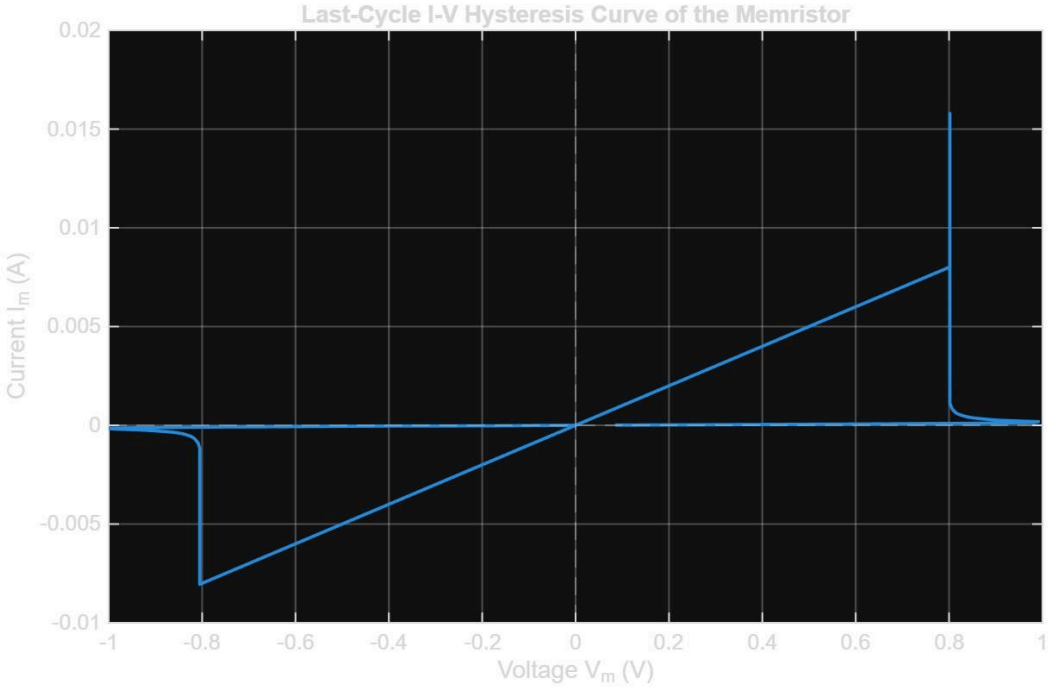


Figure 13. Simulated pinched I–V hysteresis behaviour of the flux-controlled memristor

Item	Value / description
Software	***MATLAB/Simulink/Simscape***
Input source	controlled sinusoidal voltage source
Input amplitude	$1\ V$
Input frequency	$1\ Hz$
Solver	**ode15s**
Measured signals	memristor voltage and current
Verification output	pinched $I–V$ hysteresis loop

Table 7. Memristor Subsystem Simulation Settings

3.6 MIZH-Inspired Circuit Integration and Analysis

3.6.1 MIZH Concept and Intended Architecture

The MIZH circuit was intended to move the Izhikevich model closer to a circuit form by adding a memristor into the nonlinear part of the neuron structure. In this idea, the membrane voltage v is represented by a capacitor voltage, and the recovery effect u is represented by a slower current-related branch, as shown in ***Figure 6***. The memristor is used because its resistance changes with voltage history, so it may help introduce a memory effect into the circuit. The main logic is that the input current changes the membrane voltage, the voltage changes the memristor flux $\phi(t)$, and the flux changes the memristance $M(\phi)$. This changing resistance then affects the circuit current and feeds back into the voltage response.

3.6.2 Mapping Neuron Dynamics into Circuit Elements

Fang et al. treat the membrane voltage as the capacitor voltage, and the recovery variable as the inductor current,as shown in ***Figure 6***. The nonlinear voltage term in the Izhikevich equation is then treated as a nonlinear current-voltage branch (memristor) **[6]**.

The next step was to link the MIZH circuit parameters with the original Izhikevich parameters. Following Fang et al., the circuit uses the ***relationships*** $a = R$ ***and*** $b = 1/R0$. By combining these relationships with the ***MIZH equations*** and applying ***Kirchhoff's voltage law*** to the circuit branches, the inductance and resistance values can be related back to the Izhikevich parameters a,b.

By applying ***Kirchhoff's voltage law*** to the $Rs - L$ branch

$$u_c = R_s i_L + L\frac{di_L}{dt} \tag{19}$$

which gives

$$\frac{di_L}{dt} = \frac{1}{L}u_c - \frac{R_s}{L}i_L \tag{20}$$

This has the same form as the recovery equation (16) used in the circuit model. By comparing the two equations, the mapping can be written as

$1/L = ab\ and\ Rs/L = a$ (21)

Therefore,

$L = 1/ab\ and\ Rs = 1/b$ (22)

This mapping shows how the recovery part of the neuron model can be represented by a circuit branch. For the phasic-spiking test, the parameter set from the original Izhikevich model was used: $a = 0.02$, $b = 0.25$, $c = -65$, $d = 6$. Using the mapping formulas in ***Equations (21) and (22)***, the circuit values were calculated as $Rs = 4\,M\Omega$ and $L = 200\,kH$. The capacitor value was set to $C = 1\,nF$ as an initial assumption for the circuit simulation.

3.6.3 Integration of the Memristor into the Neuron Circuit

The main circuit elements, including the resistor, capacitor, inductor and memristor branches, were assembled according to the MIZH circuit structure shown in ***Figure 6***. In this stage, the voltage source was replaced by a current source, since the MIZH model is driven by an external input current. Only $Vout$ was kept as the main output signal, because the key response to observe was the membrane-voltage-like behaviour of the circuit. The complete connection diagram is shown in ***Figure 14***. The voltage sensor was connected across the two terminals of the memristor branch to measure the voltage used for the flux calculation. Since the sensor is used only for measurement and is connected in parallel with the memristor, it does not change the equivalent circuit topology.

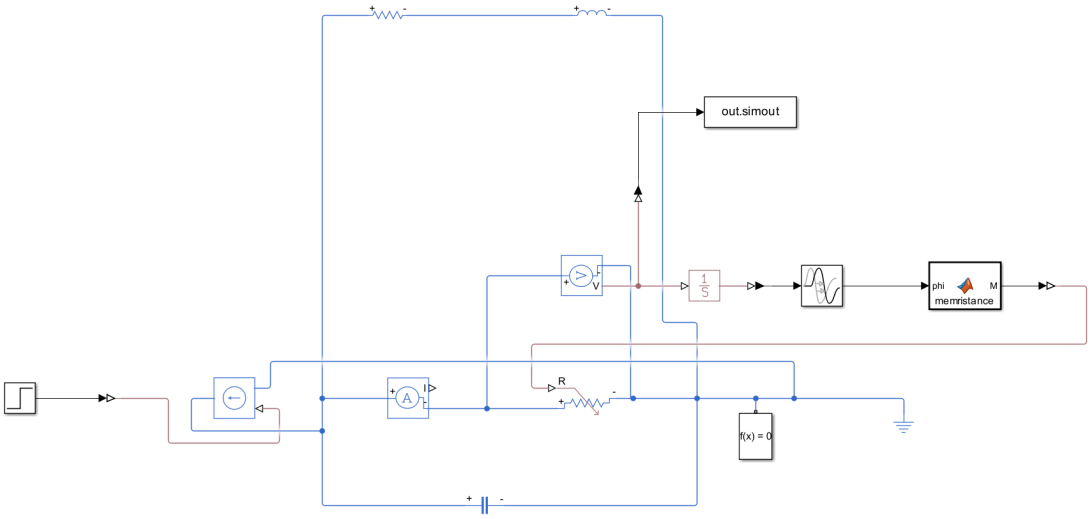


Figure 14. Simulink/Simscape implementation of the MIZH-inspired neuron circuit

The next test was to check whether the MIZH circuit could produce ***phasic spiking***. Based on the mapping discussed in Section 3.6.2, the circuit components were changed to the corresponding values. Following Fang et al.'s setting, as shown in ***Figure 15***, the input current source was set to $0.5A$, the simulation time was set to **0.2 s**, the solver was set to ***ode15s***, and the ***maximum step size*** was left as ***auto***. The aim was to see whether the circuit could reproduce the expected phasic spiking behaviour.

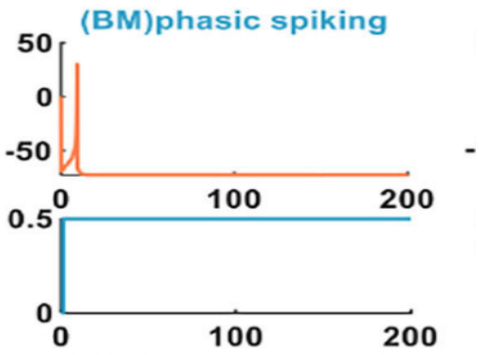


Figure 15. Phasic spiking response of the MIZH model under step input [6]

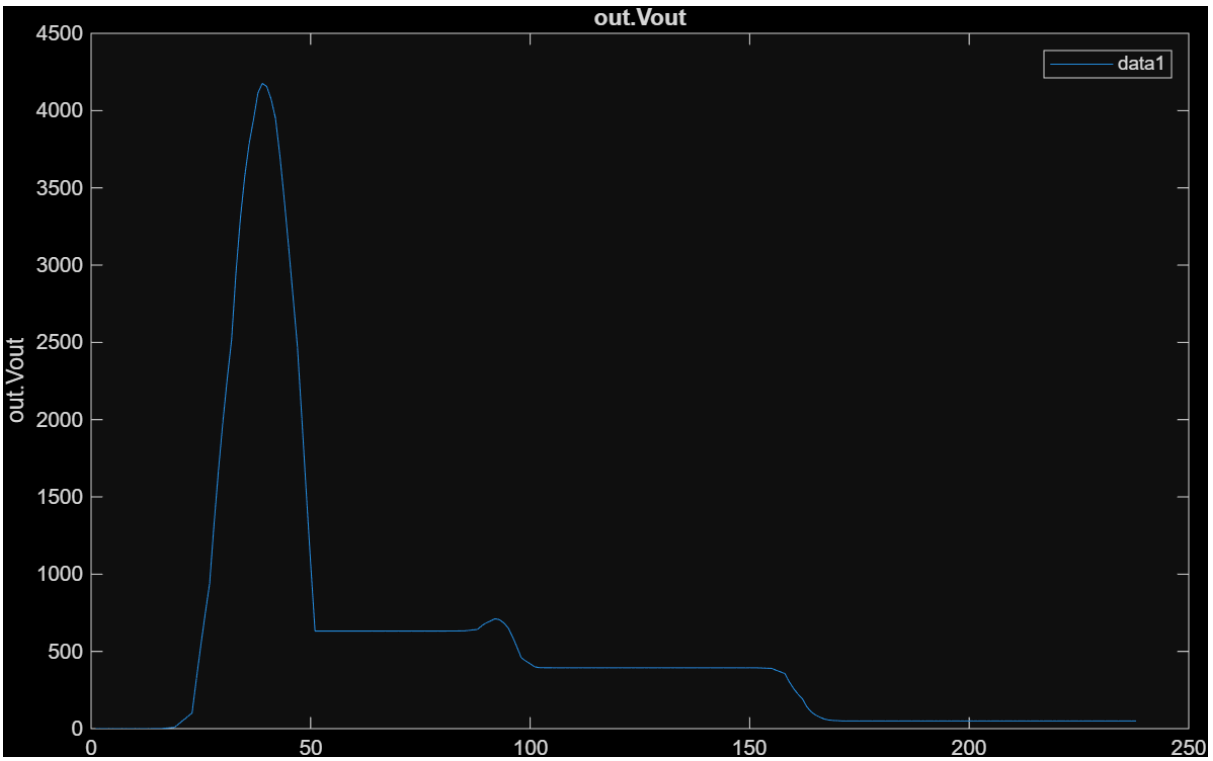


Figure 16. Unscaled voltage response of the integrated MIZH-inspired circuit

As shown in ***Figure 16,*** the integrated circuit produced a clear rise and fall in $Vout$, which means the memristor branch was affecting the circuit response. However, the voltage level was not reasonable for a neuron-like waveform. The peak value rose to more than 4000 V, which is far above the expected scale of a MIZH response. This suggests that the circuit was still suffering from a ***unit and scaling problem***. The input value used in the reference model is ***not clearly defined as a direct physical current for Simscape***, so using it without proper scaling can lead to an unrealistic voltage output.

Although the waveform has a ***rise-and-fall shape***, it is more likely to be an ***RLC–memristor transient*** than true phasic spiking. The voltage first rose to an unrealistic level and then moved through ***several plateau-like regions***, which suggests that the response was mainly caused by memristance switching and changes in the equivalent circuit resistance. A true phasic spike should show a short voltage peak followed by a return to a low resting level while the input remains applied,as shown in ***Figure 15***. This was not achieved in the present circuit. The memristor subsystem worked, and the integrated circuit showed state-dependent transient behaviour. However, the full phasic spiking behaviour was not achieved.

3.6.4 Numerical and Structural Challenges

The main reason is that the original paper provides the general MIZH circuit idea, but does not give ***all*** the implementation details needed to fully reconstruct the model in ***Simscape***. The mapping from a and b was useful because it gave circuit values for the recovery branch, such as Rs and L. However, ***the reset parameters*** c ***and*** d ***were not directly realised in the***

circuit. In the MATLAB Izhikevich model, when the voltage reaches the spike limit, the voltage is reset to c and the recovery variable is increased by d. In the Simscape circuit, this kind of reset does not happen automatically unless an extra switching or feedback path is added. Because of this, the circuit can rise in voltage, but it does not have a clear mechanism to bring the membrane voltage back down in the same way as the original model.

Another important issue is that the nonlinear voltage term in the MIZH equation was not fully reproduced as a circuit block. In the original model, this nonlinear term helps push the membrane voltage into a spike-like response. In my Simscape model, the circuit contains the capacitor, the $Rs - L$ branch and the memristor branch, ***but this does not automatically mean that the full nonlinear equation has been rebuilt.*** As a result, the circuit response is still strongly affected by normal RLC behaviour and by the change of memristance. This explains why the output showed ***a large transient peak and several plateau-like regions***, rather than a clean phasic spike.

The input scaling was also a problem. The input value used in the reference model cannot be directly treated as a physical current in Simscape. When the same value was used without scaling, the output voltage became unrealistically large, reaching thousands of volts. This shows that the mathematical model and the physical Simscape circuit are not using the same scale. Therefore, although the memristor branch was active and did affect the circuit response, the result should be interpreted as a state-dependent $RLC–memristor$ transient, not as a successful reproduction of MIZH phasic spiking.

Model quantity	**Circuit representation**	**Implemented in this project?**	**Comment**
v	capacitor voltage	yes	used as $Vout$
u	inductor current	yes	Represented by $Rs - L$ branch
(a, b)	mapped to $(Rs - L)$	yes	based on Fang et al.
c	voltage reset level	no	requires switch/reset path
d	recovery update after spike	no	requires controlled current pulse or feedback
nonlinear voltage term	nonlinear circuit branch / memristor	partly	memristor branch included, but full nonlinear equation not fully realised

Table 8. Mapping between MIZH Model Variables and Circuit Elements

Chapter 4

4. Conclusions

4.1 Summary of the Project

This project studied the move from behavioural neuron modelling to a memristive Izhikevich-inspired circuit. The first stage used MATLAB to simulate the Izhikevich and AMAT models. These results gave a useful reference for different firing behaviours, such as tonic spiking, bursting, adaptation and phasic responses. They also showed that neuron-like behaviour depends not only on the input current, but also on internal variables, reset rules and threshold dynamics.

The second stage focused on the memristor. A flux-controlled memristor subsystem was built and tested in ***Simulink/Simscape***. The subsystem showed a clear $I–V$ hysteresis curve and set/reset behaviour, so the memristor model was working at the subsystem level. This was an important result, because the memristor had to be verified before it could be used in the full MIZH-inspired circuit.

The final stage attempted to integrate the memristor into a MIZH-inspired neuron circuit. Although the memristor branch started to affect the circuit response, the full phasic spiking behaviour was not successfully reproduced. Some outputs showed rise-and-fall transient behaviour, but the voltage scale was not reasonable, and the waveform was closer to an $RLC–memristor$ transient than a true neuron spike. This showed that the step from a mathematical MIZH model to a stable Simscape circuit is not direct.

Overall, the project achieved a behavioural MATLAB baseline and a verified memristor subsystem, but the full MIZH circuit remained incomplete. The main difficulty was not only the memristor itself, but the ***full circuit mapping, input scaling, reset behaviour and feedback path*** needed to produce a real phasic-spiking response.

4.2 Main Findings

The MATLAB stage showed that Izhikevich and AMAT models are useful for understanding firing behaviour at the behavioural level. They were easy to tune and gave clear reference waveforms.

The memristor subsystem was successfully verified. It showed memory-dependent behaviour through the pinched $I–V$ hysteresis curve, although the switching was still idealised and sensitive to parameter choices.

The MIZH integration was only partly successful. The memristor could be connected into the circuit, but the output did not match the reference phasic spiking behaviour.

4.3 Limitations

This project was simulation-based only, so no hardware validation was carried out. The memristor model was also simplified and did not include real device effects such as noise, temperature change or material variation.

Another limitation is that the reset behaviour of the original Izhikevich model was not fully realised in circuit form. The parameters a and b could be linked to the recovery branch, but c and d would need extra switching or feedback logic. This made it difficult to reproduce a clean phasic spike.

4.4 Future Work

Future work should first focus on adding a clearer reset path to the MIZH-inspired circuit. In the Izhikevich model, once the voltage reaches the spike limit, the voltage is reset and the recovery variable is updated. This did not happen naturally in the current Simscape circuit. ***One possible solution is to measure the capacitor voltage*** Vc ***, compare it with a threshold, and then use a controlled switch to briefly connect the capacitor to a reset voltage or a discharge resistor.***

The recovery branch also needs further work. The current $Rs - L$ branch only gives part of the slow recovery effect. To represent the $u = u + d$ update more closely, ***a short controlled current pulse could be added to the recovery branch whenever a spike is detected.*** This would make the reset process closer to the original Izhikevich model.

Another important step is parameter scaling. The input values from the reference MIZH model should not be used directly as physical Simscape currents. ***A better method would be to choose a target voltage range first***, then estimate the input from

$$I \approx C\frac{dV}{dt} \tag{23}$$

For example, if $C = 1\,nF$ ***and the target is a*** $50\,mV$ ***rise in*** $5\,ms$ ***, the input current should be about*** $10\,nA$ ***not a large unscaled value.*** The flux scaling should also be checked. A very small voltage may not change the memristor state, while a very large voltage may make it switch too sharply. A scaling gain before the flux integrator could help the memristor reach its set/reset region without making Vc unrealistically large.

Finally, once one stable phasic-spiking-like response is achieved, the work could then be extended to tonic spiking, bursting, and small neuron networks.

Objective	Evidence	Outcome
Review neuron and memristive models	Chapter 2	Achieved

Simulate Izhikevich and MAT/AMAT models	Figures 7–11 and Appendix A	Achieved
Verify memristor subsystem	Figure 13 and Table 7	Achieved
Explore MIZH-inspired circuit integration	Figures 14–16 and Table 8	Partly achieved
Assess feasibility	Sections 3.6.4 and 4.4	Achieved through limitation analysis

Table 9. Summary of Project Achievements against Objectives

Chapter 5

Economic, Legal, Social, Ethical and Environmental Context

This project is a simulation study, so it does not directly create safety risks in the same way as a real hardware circuit. Still, the topic has a wider engineering meaning because memristive neuron circuits are related to low-power computing and neuromorphic systems.

Economically, this kind of work may be useful in the long term because hardware neuron models could reduce the cost and power needed for some computing tasks. However, the circuit in this project is still only at an early modelling stage, so it should not be seen as a finished design.

There are also some legal points to consider. The MIZH circuit idea and memristor models are based on published work, so these sources must be clearly cited. If this work were developed into a real device in the future, patent and intellectual property issues would need to be checked.

Ethically, the main point is to present the results honestly. This project does not claim to reproduce a real biological neuron, and the full MIZH circuit was not completely achieved. The work should therefore be described as a simulation-based feasibility study, not as a completed neuron circuit.

The direct environmental impact of this project is low because it was carried out in MATLAB, Simulink and Simscape. In the future, memristive hardware could help reduce energy use in computing, but real devices would still involve materials, fabrication and testing, which also have environmental costs.

Appendix A

A.1 Core MATLAB code used to generate the tonic spiking response of the Izhikevich neuron model

```
%% Izhikevich model - tonic spiking with step input

clear; clc; close all;

% Time setting
dt = 0.1;               % ms
T  = 1000;              % ms
t  = 0:dt:T;
N  = length(t);

% Tonic spiking parameters
a = 0.02;
b = 0.20;
c = -65;
d = 8;

% Input current
I = zeros(1,N);
I(t >= 200 & t <= 800) = 14;   % step input from 200 ms to 800 ms

% Initial values
v = -70 * ones(1,N);           % membrane voltage, mV
u = zeros(1,N);                % recovery variable
u(1) = b * v(1);

% Simulation
for k = 1:N-1
   if v(k) >= 30
       v(k) = 30;                    % spike peak for plotting
       v(k+1) = c;                   % reset voltage
       u(k+1) = u(k) + d;            % recovery update
   else
       dv = 0.04*v(k)^2 + 5*v(k) + 140 - u(k) + I(k);
       du = a*(b*v(k) - u(k));
       v(k+1) = v(k) + dt*dv;
       u(k+1) = u(k) + dt*du;
   end
end
```

This MATLAB script shows the implementation of the tonic spiking simulation. The key part is the use of the **Euler method** to update the membrane voltage and recovery variable, which can be seen in the update equations near the end of the code.

A.2 Core MATLAB Code for AMAT Spike Frequency Adaptation Simulation

```
%% AMAT model - spike frequency adaptation
clear; clc; close all;
% time setting
dt = 0.05;                  % ms
T  = 400;                   % ms
t  = 0:dt:T;
N  = length(t);
% membrane equation
tau_m = 10;                 % ms
R     = 50;                 % MOhm
% input current
I = zeros(1,N);
I(t >= 20) = 0.15;          % nA
% adaptive threshold parameters
omega  = 5;                 % resting threshold, mV
tau1   = 10;                % fast threshold component, ms
tau2   = 200;               % slow threshold component, ms
alpha1 = 10;                % fast jump after a spike
alpha2 = 1;                 % slow jump after a spike
% voltage-history term used in AMAT
tauV = 5;                   % ms
beta = 0.2;                 % small value for this case
% initial values
V      = zeros(1,N);
x1     = zeros(1,N);
x2     = zeros(1,N);
qV     = zeros(1,N);
theta  = zeros(1,N);
dVdt   = zeros(1,N);
spikeTimes = [];
spikeFlag  = zeros(1,N);
lastSpike  = -1e9;
minISI     = 1;             % avoids repeated numerical crossings
% simulation
for k = 1:N-1
   dVdt(k)  = (-V(k) + R*I(k)) / tau_m;
   theta(k) = omega + x1(k) + x2(k) + beta*qV(k);
   if V(k) >= theta(k) && (t(k) - lastSpike) > minISI
       spikeFlag(k) = 1;
       spikeTimes(end+1) = t(k);
       lastSpike = t(k);
       x1(k) = x1(k) + alpha1;
       x2(k) = x2(k) + alpha2;
       theta(k) = omega + x1(k) + x2(k) + beta*qV(k);
   end
   V(k+1)  = V(k)  + dt*dVdt(k);
   x1(k+1) = x1(k) + dt*(-x1(k)/tau1);
   x2(k+1) = x2(k) + dt*(-x2(k)/tau2);
   qV(k+1) = qV(k) + dt*((dVdt(k) - qV(k))/tauV);
end
dVdt(end)  = (-V(end) + R*I(end)) / tau_m;
theta(end) = omega + x1(end) + x2(end) + beta*qV(end);
%% grouped burst detection
burstMask = zeros(1,N);
burstCount = 0;
isiLimit = 6;               % ms
if length(spikeTimes) > 1
   groupStart = spikeTimes(1);
   groupN = 1;
   for n = 2:length(spikeTimes)
       if spikeTimes(n) - spikeTimes(n-1) <= isiLimit
           groupN = groupN + 1;
       else
           if groupN > 1
               burstCount = burstCount + 1;
               burstMask(t >= groupStart & t <= spikeTimes(n-1)) = 1;
           end
```

```
            groupStart = spikeTimes(n);
            groupN = 1;
        end
    end
    if groupN > 1
        burstCount = burstCount + 1;
        burstMask(t >= groupStart & t <= spikeTimes(end)) = 1;
    end
end
```

This MATLAB script was used to generate the spike-frequency adaptation case in the AMAT-style adaptive-threshold model. The membrane voltage is updated using the Euler method, while the firing threshold is updated by **fast, slow** and **voltage-history-dependent components**. The burst-counting part groups neighbouring spike events when their inter-spike interval is shorter than a selected limit, so several close spikes are counted as one burst instead of being treated as separate threshold crossings.

A.3 Additional Izhikevich Model Simulation Results

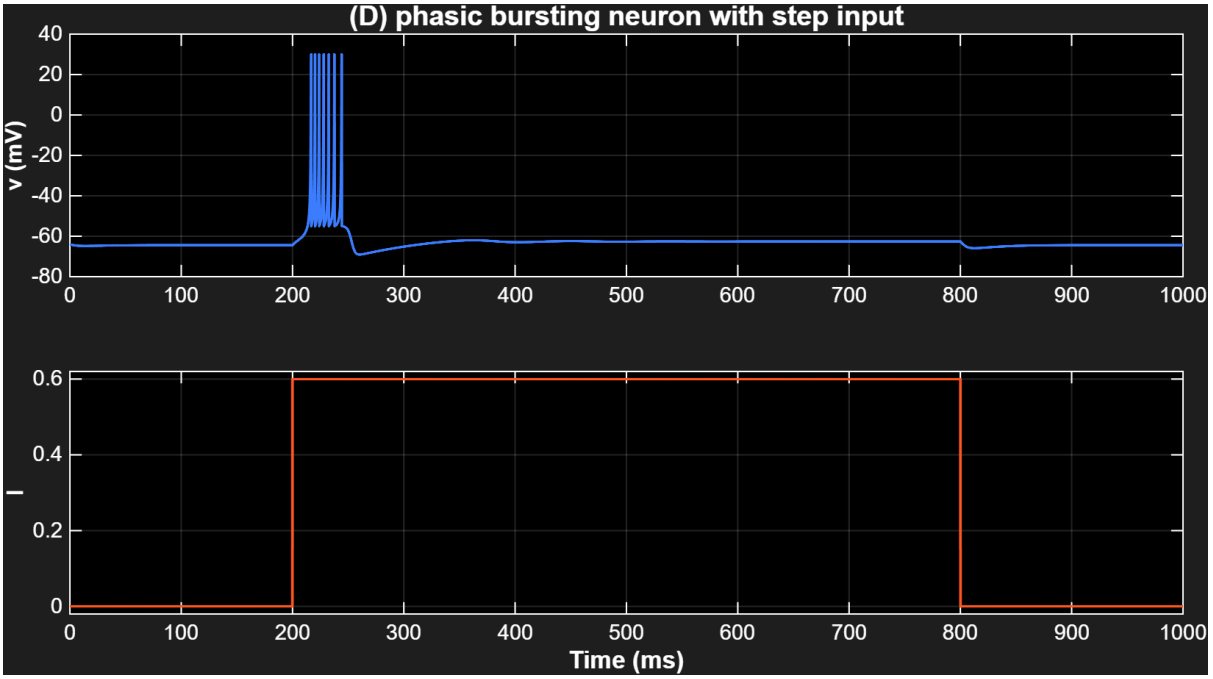


Figure A.1 Phasic bursting behaviour of an Izhikevich neuron under step current input

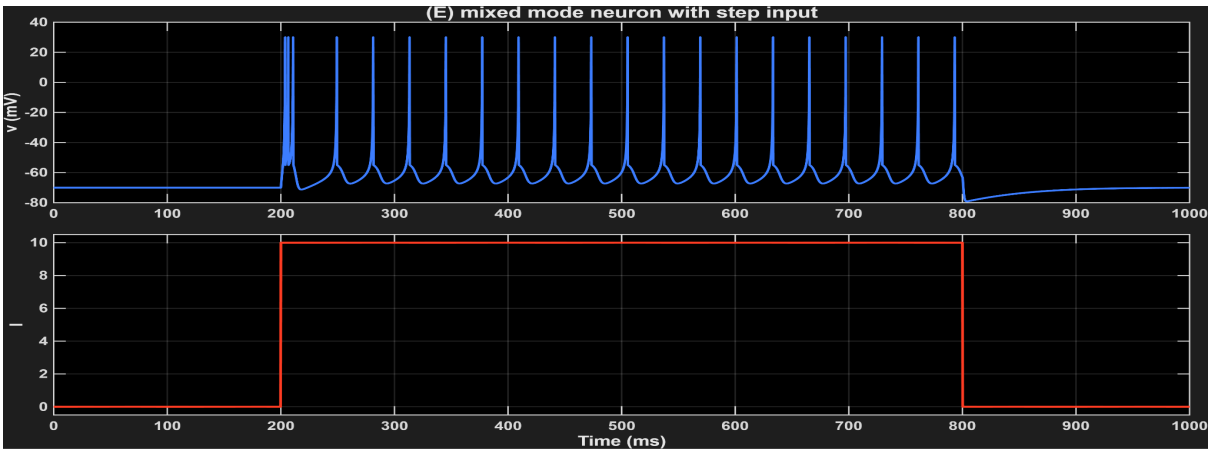


Figure A.2 Mixed mode behaviour of an Izhikevich neuron under step current input

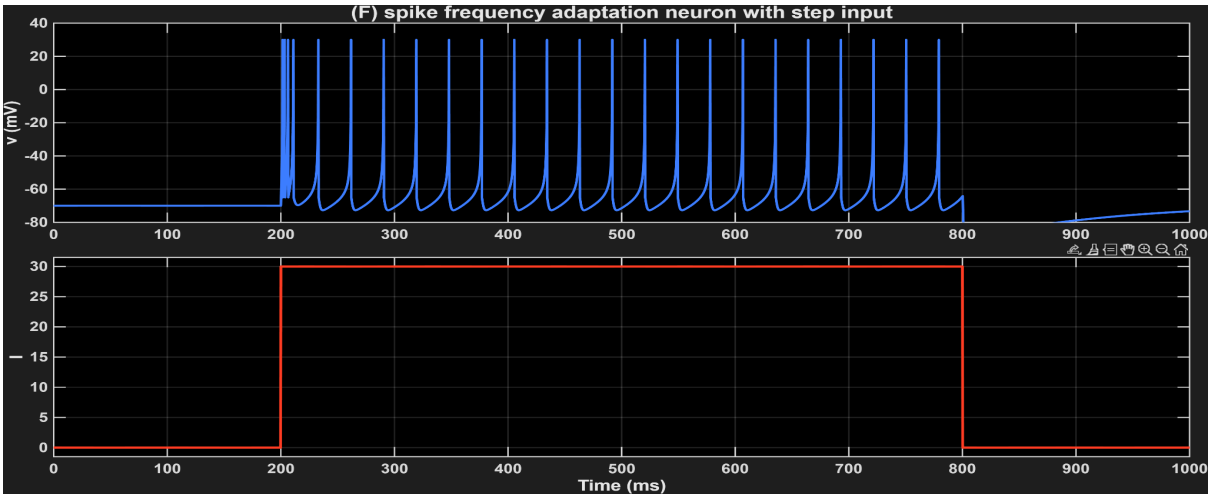


***Figure A.3** Spike frequency adaptation behaviour of an Izhikevich neuron under step current input*

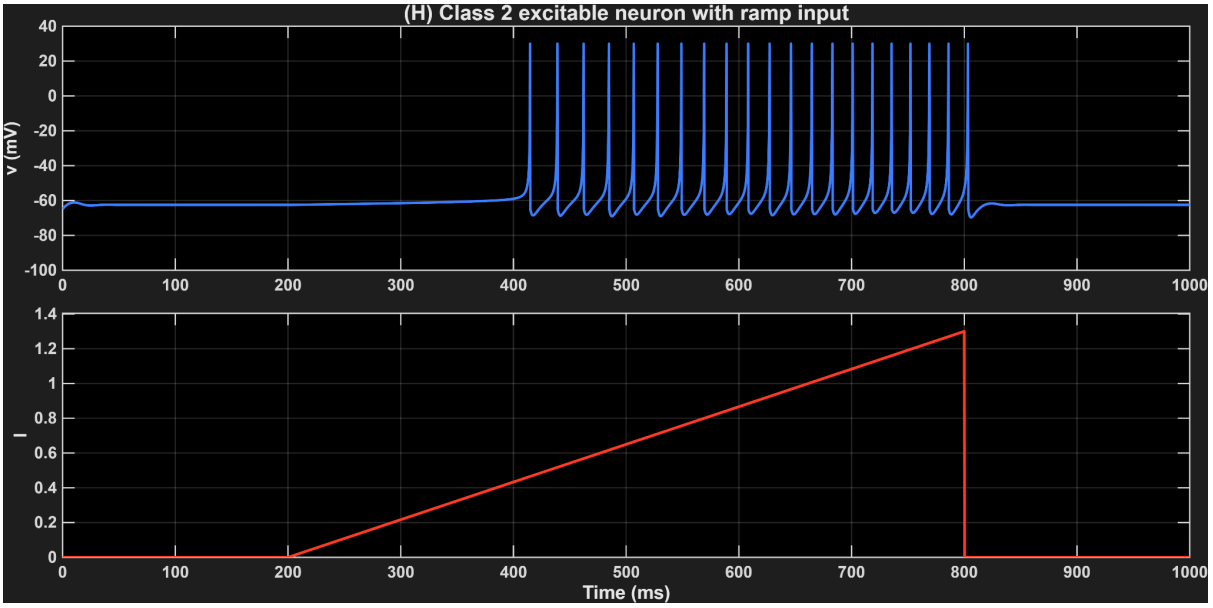


Figure A.4 Class 2 behaviour of an Izhikevich neuron under ramp current input

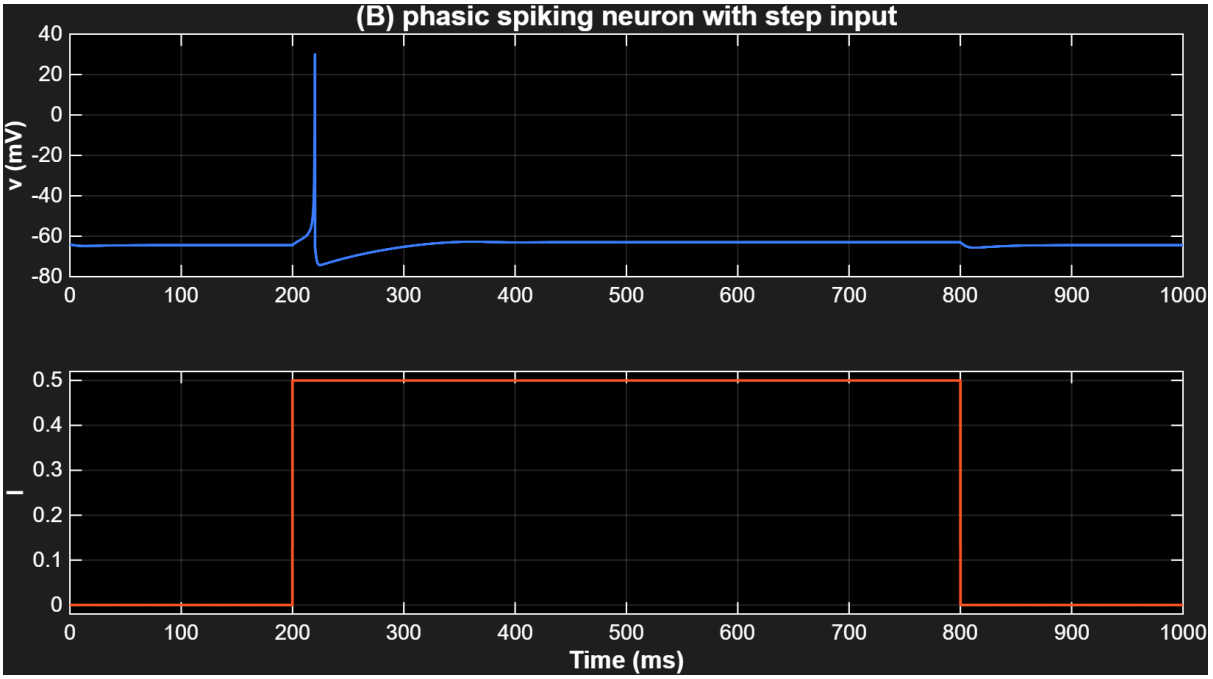


Figure A.5 Phasic spiking behaviour of an Izhikevich neuron under step current input

A.4 Additional MAT/AMAT Model Simulation Results

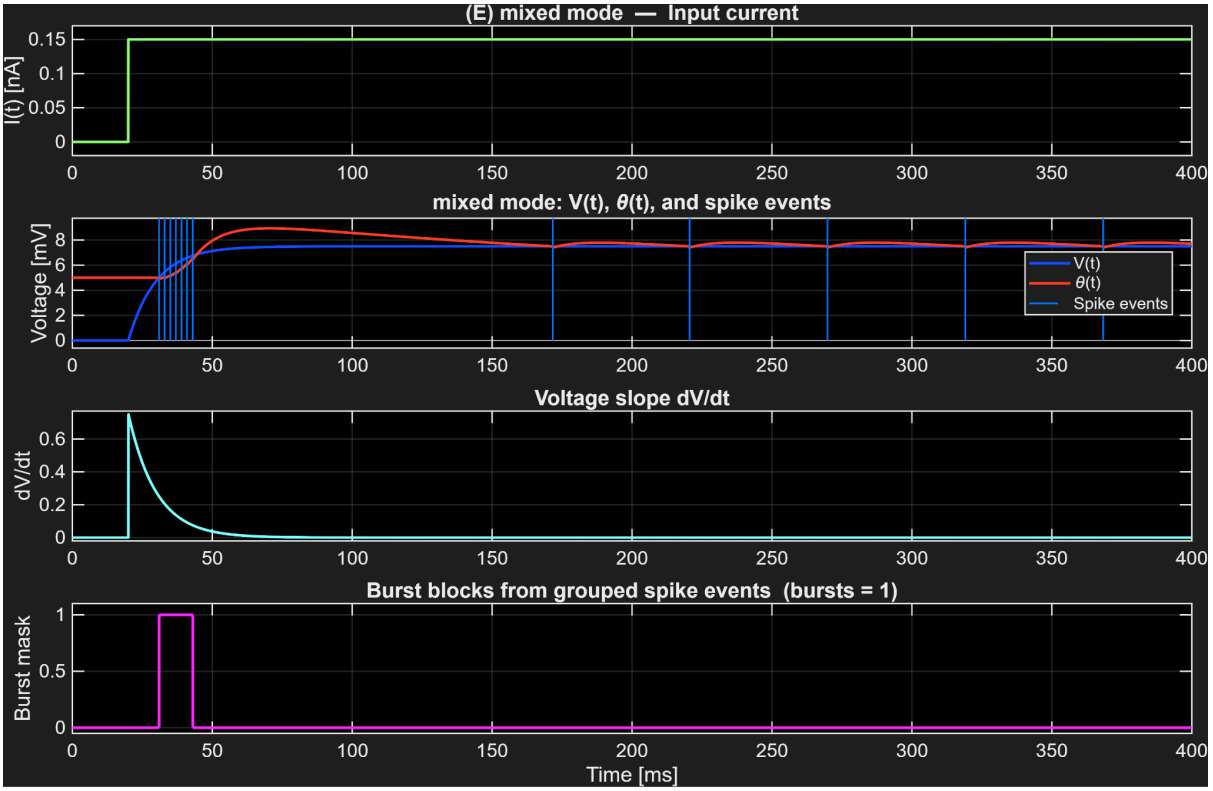


Figure A.6 Mixed mode result using grouped spike-event detection

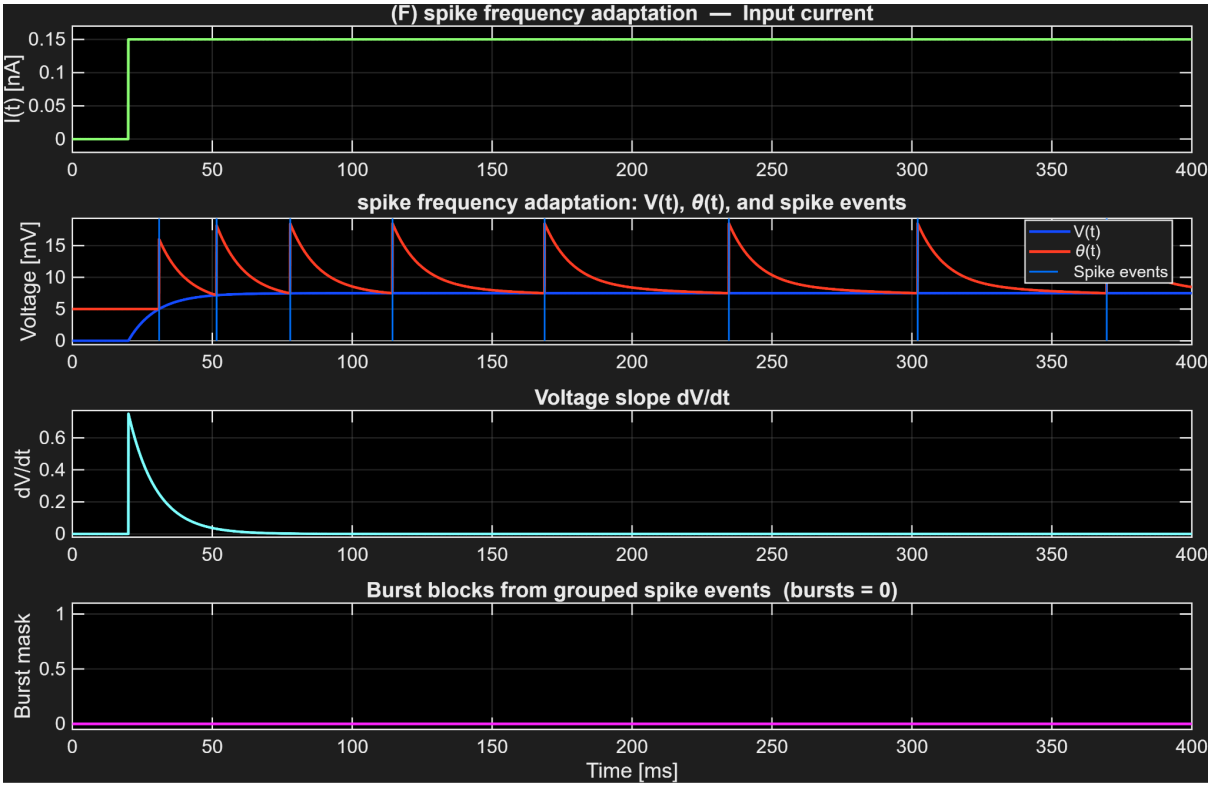


Figure A.7 Spike frequency adaptation result using grouped spike-event detection

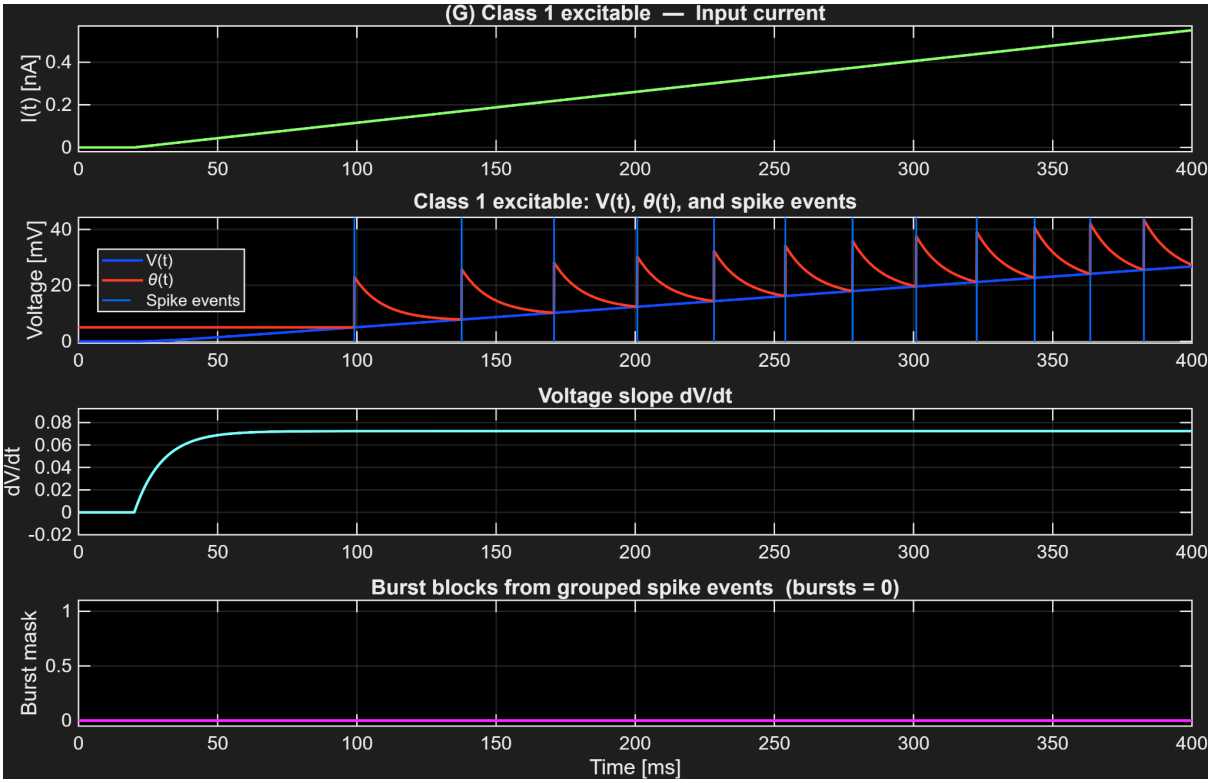


Figure A.8 Class 1 result using grouped spike-event detection

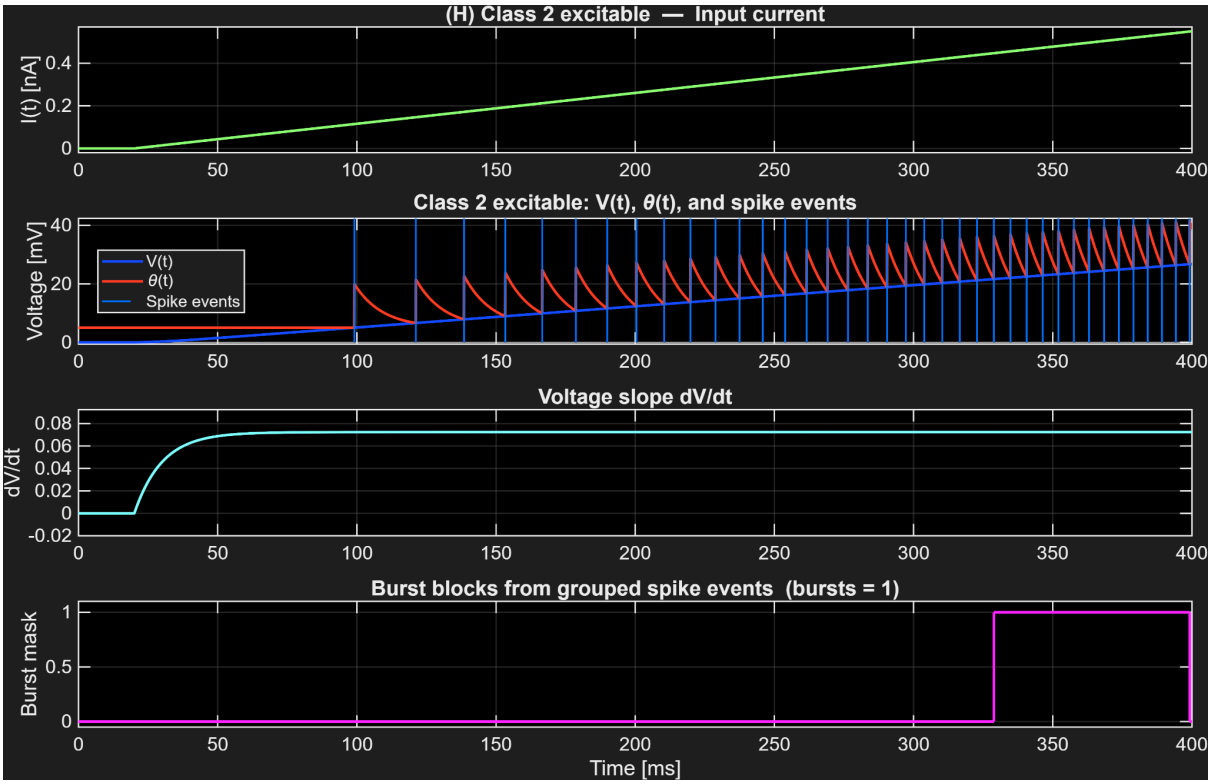


Figure A.9 Class 2 result using grouped spike-event detection

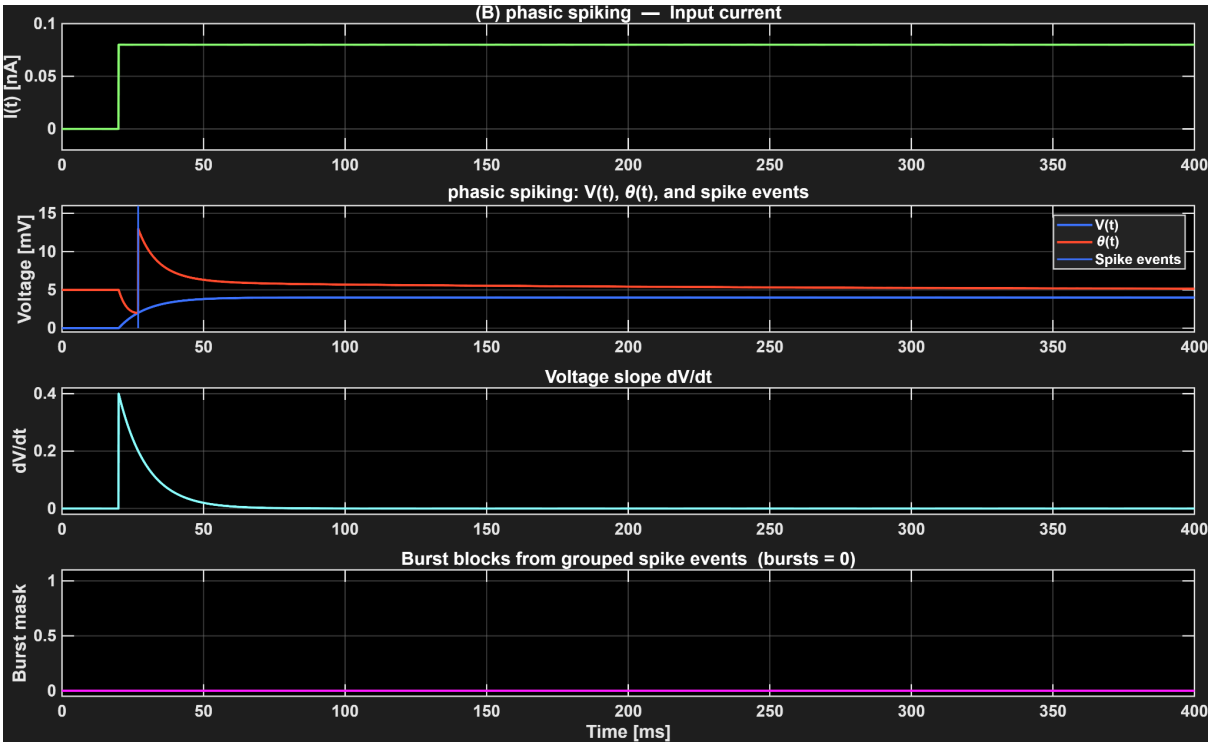


Figure A.10 Phasic spiking result using grouped spike-event detection

A.5 MATLAB Function for Calculating Flux-Dependent Memristance

```
function M = memristance(phi)
if phi < -0.75
   M = 20000;
elseif phi < 0.25
   x = 1e8 - 3.98e8 * phi;
   if x < 0
       x = 0;
   end
   M = sqrt(x);
else
   M = 100;
end
```

References

[1] E. M. Izhikevich, “Simple model of spiking neurons,” *IEEE Transactions on Neural Networks*, vol. 14, no. 6, pp. 1569–1572, 2003, doi: 10.1109/TNN.2003.820440.

[2] E. M. Izhikevich, “Which model to use for cortical spiking neurons?,” *IEEE Transactions on Neural Networks*, vol. 15, no. 5, pp. 1063–1070, 2004, doi: 10.1109/TNN.2004.832719.

[3] S. Yamauchi, H. Kim, and S. Shinomoto, “Elemental spiking neuron model for reproducing diverse firing patterns and predicting precise firing times,” *Frontiers in Computational Neuroscience*, vol. 5, Art. no. 42, pp. 1–15, 2011, doi: 10.3389/fncom.2011.00042.

[4] T. Heiberg, B. Kriener, T. Tetzlaff, G. T. Einevoll, and H. E. Plesser, “Firing-rate models for neurons with a broad repertoire of spiking behaviors,” *Journal of Computational Neuroscience*, vol. 45, pp. 103–132, 2018, doi: 10.1007/s10827-018-0693-9.

[5] A. L. Hodgkin and A. F. Huxley, “A quantitative description of membrane current and its application to conduction and excitation in nerve,” *The Journal of Physiology*, vol. 117, no. 4, pp. 500–544, 1952, doi: 10.1113/jphysiol.1952.sp004764.

[6] X. Fang, S. Duan, and L. Wang, “Memristive Izhikevich spiking neuron model and its application in oscillatory associative memory,” *Frontiers in Neuroscience*, vol. 16, Art. no. 885322, 2022, doi: 10.3389/fnins.2022.885322.